

STAR RX72N - rx_harq Library
1.0.0

Generated by Doxygen 1.16.1

1 Test List	1
2 Directory Hierarchy	3
2.1 Directories	3
3 File Index	5
3.1 File List	5
4 Directory Documentation	7
4.1 libs/rx_harq/inc Directory Reference	7
4.2 libs Directory Reference	7
4.3 libs/rx_harq Directory Reference	8
4.4 libs/rx_harq/src Directory Reference	8
5 File Documentation	9
5.1 libs/rx_harq/inc/rx_harq.h File Reference	9
5.1.1 Detailed Description	11
5.1.1.1 HARQ Protocol Overview	11
5.1.1.2 Chase Combining Theory	11
5.1.1.3 Protocol Sequence	12
5.1.1.4 HARQ State Machine	13
5.1.1.5 Performance Characteristics	13
5.1.1.6 Features	14
5.1.2 Data Structure Documentation	15
5.1.2.1 struct rx_chase_combiner_t	15
5.1.2.2 struct rx_harq_handle_t	16
5.1.2.3 struct rx_harq_config_t	17
5.1.2.4 struct rx_harq_decode_params_t	18
5.1.3 Enumeration Type Documentation	18
5.1.3.1 rx_harq_params_t	18
5.1.3.2 rx_harq_state_t	19
5.1.4 Function Documentation	19
5.1.4.1 rx_chase_combiner_add()	19
5.1.4.2 rx_chase_combiner_can_add()	20
5.1.4.3 rx_chase_combiner_combined()	21
5.1.4.4 rx_chase_combiner_count()	22
5.1.4.5 rx_chase_combiner_deinit()	23
5.1.4.6 rx_chase_combiner_init()	24
5.1.4.7 rx_chase_combiner_reset()	25
5.1.4.8 rx_harq_can_retry()	26
5.1.4.9 rx_harq_decode()	26
5.1.4.10 rx_harq_deinit()	28
5.1.4.11 rx_harq_encode()	29
5.1.4.12 rx_harq_get_retry_count()	30
5.1.4.13 rx_harq_get_state()	30

5.1.4.14 rx_harq_init()	31
5.1.4.15 rx_harq_reset()	32
5.2 rx_harq.h	33
5.3 libs/rx_harq/src/rx_harq.c File Reference	38
5.3.1 Detailed Description	39
5.3.1.1 Implementation Architecture	40
5.3.1.2 Chase Combining Algorithm	40
5.3.1.3 Overflow Prevention	40
5.3.1.4 Protocol Flow	41
5.3.2 Enumeration Type Documentation	42
5.3.2.1 bit_constants_t	42
5.3.2.2 harq_bool_t	42
5.3.2.3 harq_cfg_sentinel_t	43
5.3.2.4 harq_fec_constants_t	43
5.3.3 Function Documentation	44
5.3.3.1 internal_handle_fec_result()	44
5.3.3.2 internal_soft_to_hard()	45
5.3.3.3 rx_chase_combiner_add()	45
5.3.3.4 rx_chase_combiner_can_add()	47
5.3.3.5 rx_chase_combiner_combined()	47
5.3.3.6 rx_chase_combiner_count()	48
5.3.3.7 rx_chase_combiner_deinit()	49
5.3.3.8 rx_chase_combiner_init()	49
5.3.3.9 rx_chase_combiner_reset()	50
5.3.3.10 rx_harq_can_retry()	51
5.3.3.11 rx_harq_decode()	52
5.3.3.12 rx_harq_deinit()	53
5.3.3.13 rx_harq_encode()	54
5.3.3.14 rx_harq_get_retry_count()	55
5.3.3.15 rx_harq_get_state()	55
5.3.3.16 rx_harq_init()	56
5.3.3.17 rx_harq_reset()	57
5.4 rx_harq.c	58

Chapter 1

Test List

File [rx_harq.h](#)

test_rx_harq.c Unit tests for HARQ protocol

Chapter 2

Directory Hierarchy

2.1 Directories

inc	7
rx_harq.h	9
libs	7
rx_harq	8
inc	7
rx_harq.h	9
src	8
rx_harq.c	38
rx_harq	8
inc	7
rx_harq.h	9
src	8
rx_harq.c	38
src	8
rx_harq.c	38

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

libs/rx_harq/inc/ rx_harq.h	
Hybrid Automatic Repeat Request (HARQ) Protocol for RX72N	9
libs/rx_harq/src/ rx_harq.c	
Hybrid Automatic Repeat Request (HARQ) Protocol Implementation	38

Chapter 4

Directory Documentation

4.1 libs/rx_harq/inc Directory Reference

Directory dependency graph for inc:

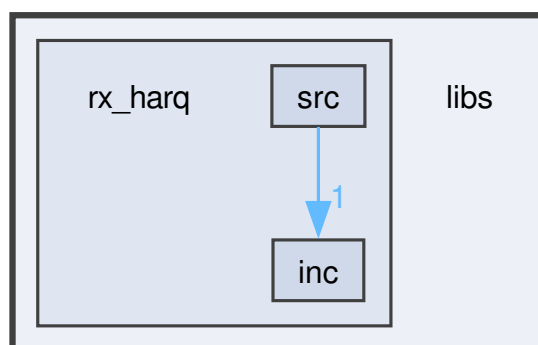


Files

- file [rx_harq.h](#)
Hybrid Automatic Repeat Request (HARQ) Protocol for RX72N.

4.2 libs Directory Reference

Directory dependency graph for libs:

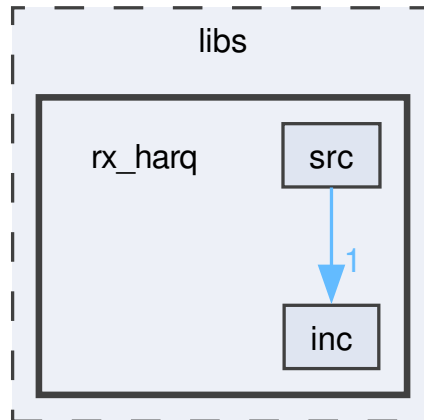


Directories

- directory [rx_harq](#)

4.3 libs/rx_harq Directory Reference

Directory dependency graph for rx_harq:

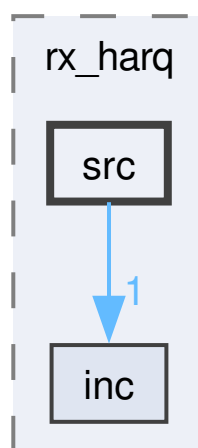


Directories

- directory [inc](#)
- directory [src](#)

4.4 libs/rx_harq/src Directory Reference

Directory dependency graph for src:



Files

- file [rx_harq.c](#)

Hybrid Automatic Repeat Request (HARQ) Protocol Implementation.

Chapter 5

File Documentation

5.1 libs/rx_harq/inc/rx_harq.h File Reference

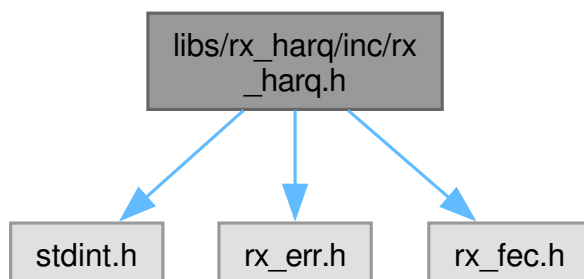
Hybrid Automatic Repeat Request (HARQ) Protocol for RX72N.

```
#include <stdint.h>
```

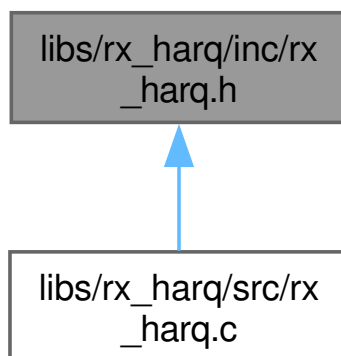
```
#include "rx_err.h"
```

```
#include "rx_fec.h"
```

Include dependency graph for rx_harq.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [rx_chase_combiner_t](#)
Chase Combiner for soft bit accumulation.
- struct [rx_harq_handle_t](#)
HARQ protocol handle.
- struct [rx_harq_config_t](#)
HARQ configuration.
- struct [rx_harq_decode_params_t](#)
HARQ decode parameters.

Enumerations

- enum [rx_harq_params_t](#) : [uint16_t](#) {
 [k_harq_max_payload](#) = 1024 , [k_harq_default_retries](#) = 3 , [k_harq_default_combines](#) = 3 ,
 [k_harq_soft_buffer_size](#) = 16400 ,
 [k_harq_survivors_buffer_size](#) = [k_harq_soft_buffer_size](#) / 2U }
HARQ protocol parameters.
- enum [rx_harq_state_t](#) : [uint8_t](#) { [k_harq_state_idle](#) = 0 , [k_harq_state_waiting_ack](#) = 1 ,
 [k_harq_state_combining](#) = 2 , [k_harq_state_error](#) = 3 }
HARQ state machine states.

Functions

- [rx_err_t rx_chase_combiner_init](#) ([rx_chase_combiner_t](#) *combiner, [uint8_t](#) max_combines)
Initialize Chase Combiner.
- [rx_err_t rx_chase_combiner_deinit](#) ([rx_chase_combiner_t](#) *combiner)
Deinitialize Chase Combiner.
- [rx_err_t rx_chase_combiner_add](#) ([rx_chase_combiner_t](#) *combiner, const [rx_soft_bit_t](#) *soft_bits, [uint32_t](#) len)
Add soft bits from a transmission attempt.
- [rx_err_t rx_chase_combiner_combined](#) (const [rx_chase_combiner_t](#) *combiner, [rx_soft_bit_t](#) *output, [uint32_t](#) *len)
Get combined soft bits for decoding.
- [rx_err_t rx_chase_combiner_reset](#) ([rx_chase_combiner_t](#) *combiner)
Reset combiner for new frame.
- [bool rx_chase_combiner_can_add](#) (const [rx_chase_combiner_t](#) *combiner)
Check if more transmissions can be combined.
- [uint8_t rx_chase_combiner_count](#) (const [rx_chase_combiner_t](#) *combiner)
Get number of combined transmissions.
- [rx_err_t rx_harq_init](#) ([rx_harq_handle_t](#) *harq, const [rx_harq_config_t](#) *config)
Initialize HARQ handle.
- [rx_err_t rx_harq_deinit](#) ([rx_harq_handle_t](#) *harq)
Deinitialize HARQ handle.
- [rx_harq_state_t rx_harq_get_state](#) (const [rx_harq_handle_t](#) *harq)
Get current HARQ state.
- [rx_err_t rx_harq_reset](#) ([rx_harq_handle_t](#) *harq)
Reset HARQ for new transaction.
- [rx_err_t rx_harq_encode](#) (const [rx_harq_handle_t](#) *harq, const [uint8_t](#) *payload, [uint32_t](#) payload_len, [uint8_t](#) *output, [uint32_t](#) output_size, [uint32_t](#) *output_len)
Encode payload with FEC (if enabled).

- `rx_err_t rx_harq_decode(rx_harq_handle_t *harq, const rx_harq_decode_params_t *params, uint8_t *output, uint32_t *output_len)`
Decode soft bits with combining and FEC.
- `uint8_t rx_harq_get_retry_count(const rx_harq_handle_t *harq)`
Get current retry count.
- `bool rx_harq_can_retry(const rx_harq_handle_t *harq)`
Check if more retries are available.

5.1.1 Detailed Description

Hybrid Automatic Repeat Request (HARQ) Protocol for RX72N.

Implements Chase Combining HARQ Type I for reliable communication over lossy channels. Combines FEC (Forward Error Correction) with ARQ (Automatic Repeat Request) to achieve both error correction and detection.

5.1.1.1 HARQ Protocol Overview

HARQ merges FEC and ARQ:

- FEC: Corrects random bit errors via convolutional coding
- ARQ: Requests retransmission when FEC fails
- Chase Combining: Accumulates soft bits from multiple transmissions

5.1.1.2 Chase Combining Theory

When a transmission fails to decode, instead of discarding it, the receiver:

1. Stores soft bits from failed attempt
2. Requests retransmission (NACK)
3. Adds new soft bits to accumulated values (element-wise)
4. Decodes combined soft bits (higher SNR -> better chance)

5.1.1.2.1 Mathematical Model

For n transmissions of the same data, the combined soft bit is:

$$s_{\text{combined}}[i] = \sum_{k=1}^n s_k[i]$$

This provides diversity gain: independent noise samples are averaged out.

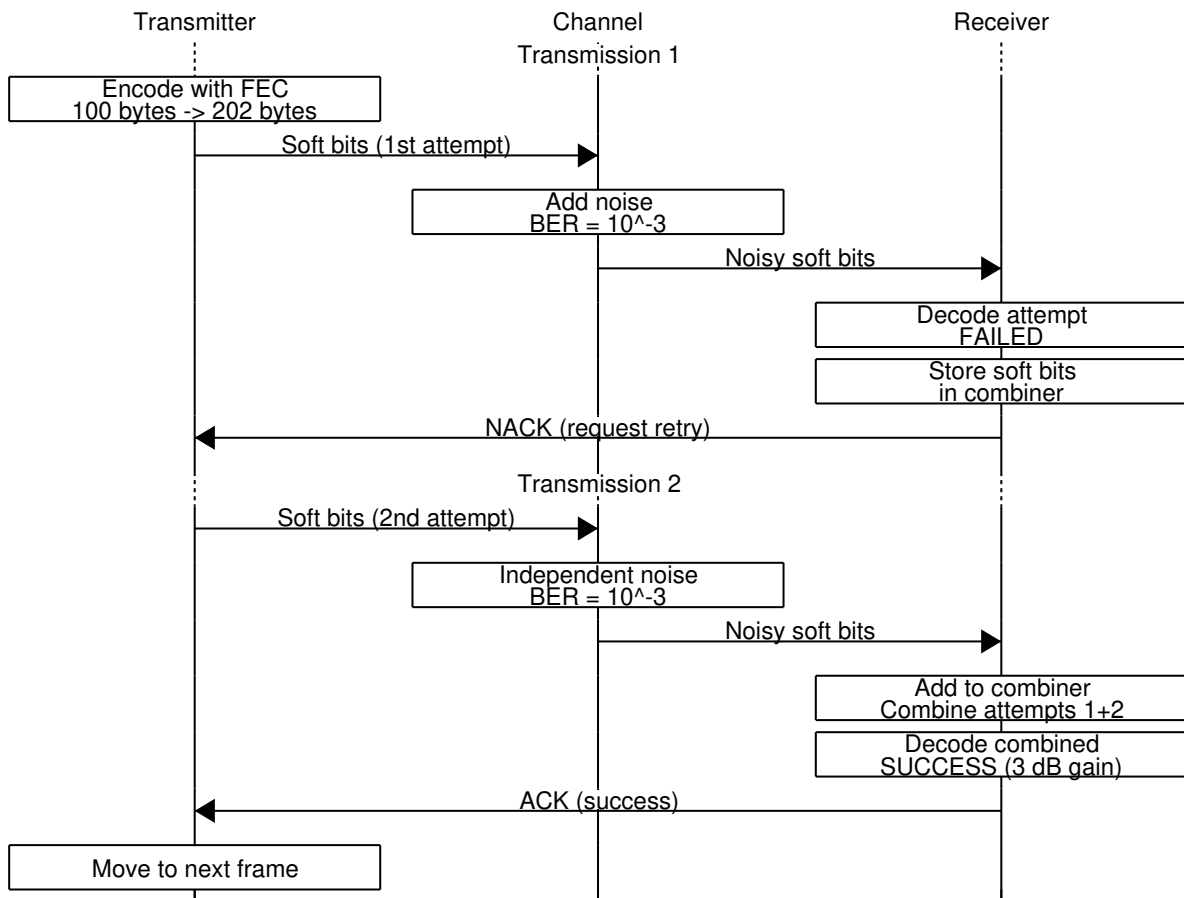
5.1.1.2.2 SNR Improvement

Assuming independent noise across transmissions, SNR improves as:

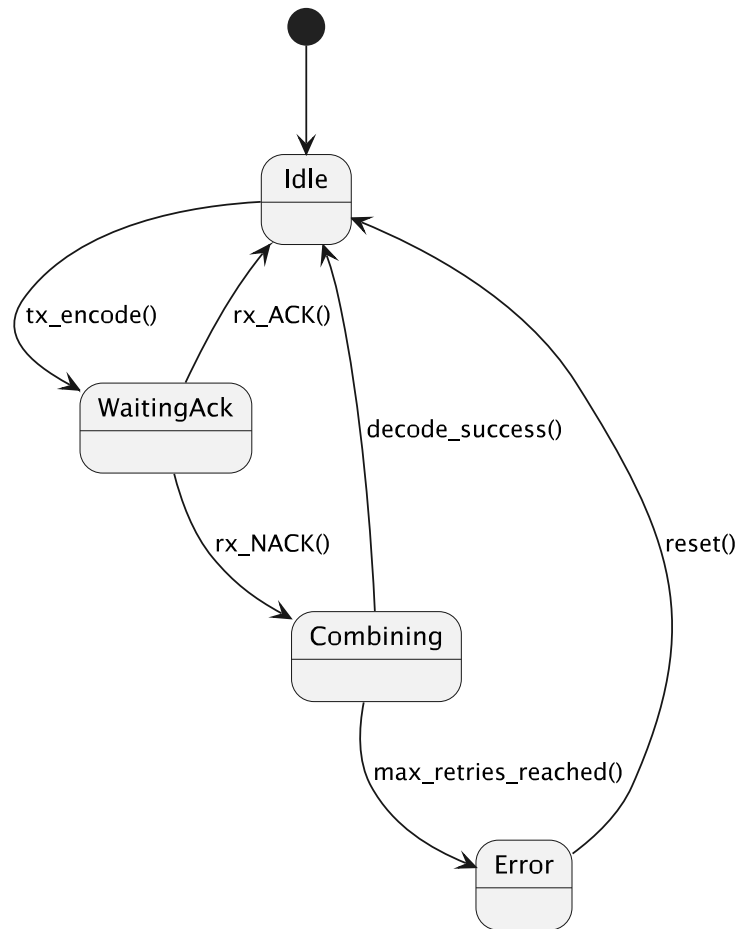
$$\text{SNR}_{\text{combined}} \approx n \cdot \text{SNR}_{\text{single}}$$

In dB: $10 \log_{10}(n)$ gain (e.g., 3 dB for 2 transmissions, 4.8 dB for 3).

5.1.1.3 Protocol Sequence



5.1.1.4 HARQ State Machine



State Descriptions:

- Idle: Ready for new transmission, no pending frames
- WaitingAck: Waiting for ACK/NACK from receiver
- Combining: Accumulating retransmissions, attempting decode
- Error: Unrecoverable error (exceeded retry limit)

Transition Guards:

- WaitingAck -> Combining: Only if `retry_count < max_retries`
- Combining -> Error: When `retry_count >= max_retries` AND decode still fails

5.1.1.5 Performance Characteristics

Metric	Value
Throughput (1 tx)	100 / 202 ~ 49.5% (FEC overhead)

Metric	Value
Throughput (2 tx avg)	100 / 404 ~ 24.8% (2x retransmissions)
Latency (1 tx)	~1 ms encode + transmit
Latency (2 tx)	~2-5 ms (includes RTT for NACK)
Memory (per handle)	~82 KB (dominated by FEC decoder)

5.1.1.6 Features

- [OK] Chase Combining: Soft bit accumulation (int16 to prevent overflow)
- [OK] FEC Integration: Optional K=7 rate 1/2 convolutional code
- [OK] Configurable Retries: 1-255 attempts (default: 3)
- [OK] Static Allocation: Zero malloc (NASA Rule 3)
- [OK] Thread Safety: Safe if handles not shared

Hardware Requirements

Resource	Usage
CPU	RX72N @ 240 MHz
RAM	~82 KB per HARQ handle
Flash	~8 KB (code size with FEC)

Module Dependencies

- `rx_fec.h`: FEC encoder/decoder
- `rx_err.h`: Error codes

NASA Power of 10 Compliance

- Rule 1: [OK] No recursion, goto, setjmp/longjmp
- Rule 2: [OK] All loops bounded (`k_harq_soft_buffer_size`)
- Rule 3: [OK] Zero dynamic allocation
- Rule 4: [OK] Functions ≤ 60 lines
- Rule 5: [OK] All functions have ≥ 2 pre/postconditions

Note

Thread Safety: HARQ handles are NOT thread-safe. Caller must provide synchronization if sharing handles between threads.

Warning

Soft bit accumulation uses int16 (range ± 32768). With max 3 combines and soft bits ± 127 , accumulator saturates at ± 381 (no overflow).

See also

[star-gateway/internal/harq/](#) Go reference implementation
[star-gateway/internal/fec/combiner.go](#) Chase Combiner algorithm
[docs/sections/01_nanopb_protocol.tex](#) Protocol specification
"Hybrid ARQ Schemes" by S. Lin & D. Costello

Author

Locked, Inc.

Date

2026-01-27

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Since

Version 1.0.0

Version

1.0.0

[Test](#) [test_rx_harq.c](#) Unit tests for HARQ protocol

Definition in file [rx_harq.h](#).

5.1.2 Data Structure Documentation

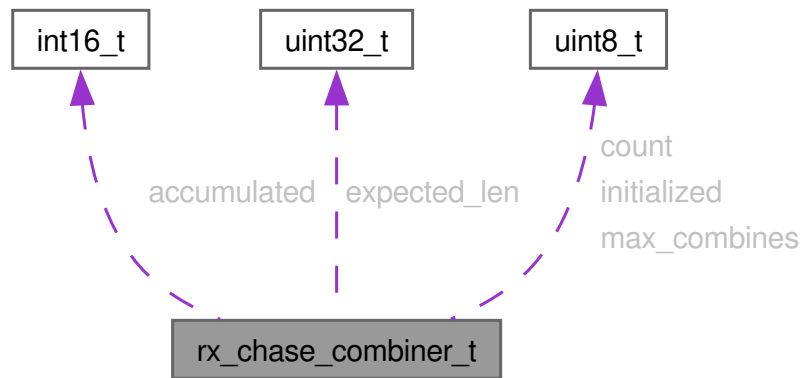
5.1.2.1 struct rx_chase_combiner_t

Chase Combiner for soft bit accumulation.

Stores soft bits from failed transmission attempts and combines them element-wise (addition) before decoding. Uses int16 accumulators to prevent overflow.

Definition at line [202](#) of file [rx_harq.h](#).

Collaboration diagram for rx_chase_combiner_t:



Data Fields

<code>int16_t</code>	<code>accumulated[k_harq_soft_buffer_size]</code>	Accumulated soft bits
<code>uint8_t</code>	<code>count</code>	Transmissions combined
<code>uint32_t</code>	<code>expected_len</code>	Expected soft bits len
<code>uint8_t</code>	<code>initialized</code>	Non-zero if initialized
<code>uint8_t</code>	<code>max_combines</code>	Max combining attempts

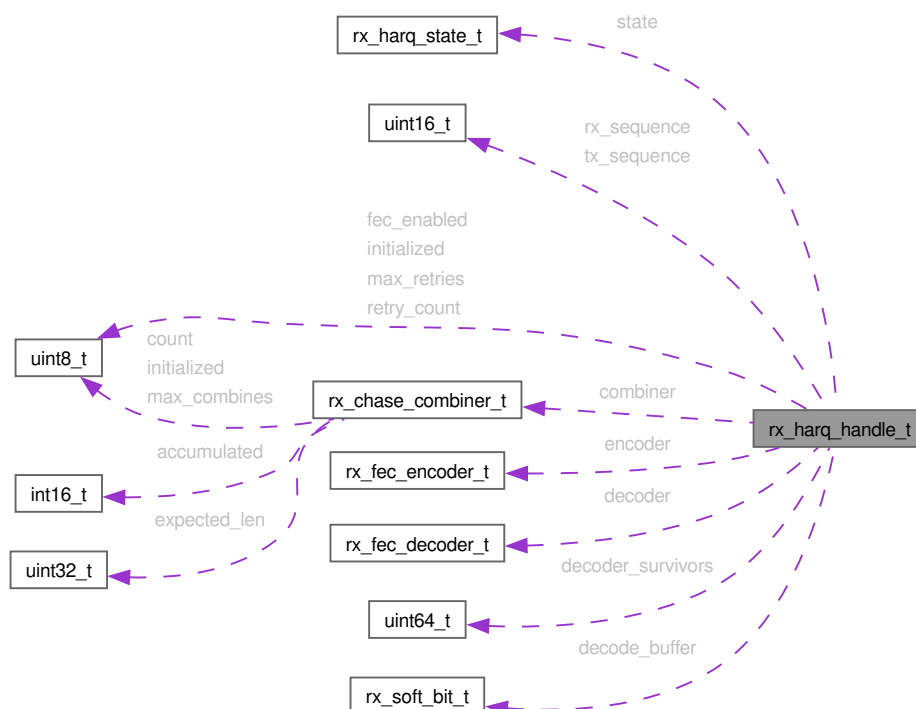
5.1.2.2 struct `rx_harq_handle_t`

HARQ protocol handle.

Contains all state for HARQ operation including FEC encoder/decoder and Chase Combiner.

Definition at line 301 of file `rx_harq.h`.

Collaboration diagram for `rx_harq_handle_t`:



Data Fields

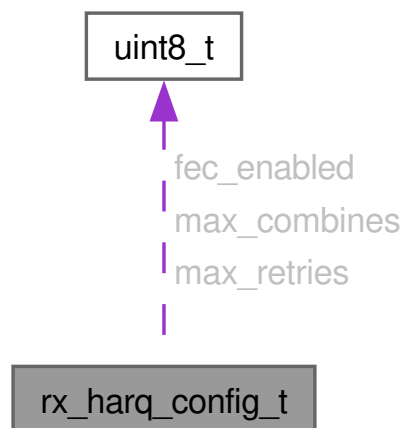
rx_chase_combiner_t	combiner	Chase Combiner
rx_soft_bit_t	decode_buffer[k_harq_soft_buffer_size]	Combined soft bits buf
rx_fec_decoder_t	decoder	FEC decoder
uint64_t	decoder_survivors[k_harq_survivors_buffer_size]	Decoder buf
rx_fec_encoder_t	encoder	FEC encoder
uint8_t	fec_enabled	Non-zero if FEC is enabled
uint8_t	initialized	Non-zero if initialized
uint8_t	max_retries	Maximum retries allowed
uint8_t	retry_count	Current retry count
uint16_t	rx_sequence	RX sequence number
rx_harq_state_t	state	Current HARQ state
uint16_t	tx_sequence	TX sequence number

5.1.2.3 struct rx_harq_config_t

HARQ configuration.

Definition at line 319 of file [rx_harq.h](#).

Collaboration diagram for rx_harq_config_t:



Data Fields

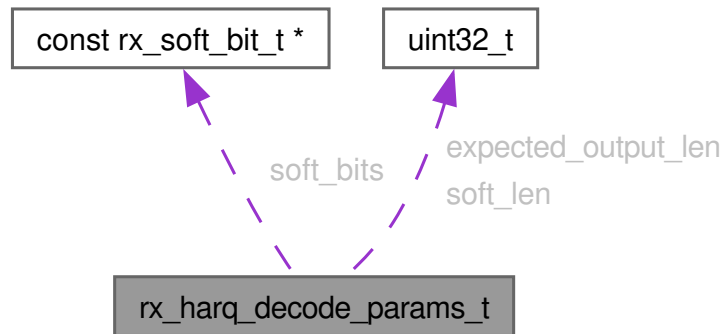
uint8_t	fec_enabled	Non-zero to enable FEC encoding/decoding
uint8_t	max_combines	Maximum combining attempts (0 = default)
uint8_t	max_retries	Maximum transmission attempts (0 = default)

5.1.2.4 struct rx_harq_decode_params_t

HARQ decode parameters.

Definition at line 328 of file [rx_harq.h](#).

Collaboration diagram for rx_harq_decode_params_t:



Data Fields

uint32_t	expected_output_len	Expected decoded length
const rx_soft_bit_t *	soft_bits	Received soft bits
uint32_t	soft_len	Number of soft bits

5.1.3 Enumeration Type Documentation

5.1.3.1 rx_harq_params_t

enum [rx_harq_params_t](#) : uint16_t

HARQ protocol parameters.

Enumerator

k_harq_max_payload	Maximum payload size in bytes
k_harq_default_retries	Default maximum retries
k_harq_default_combines	Default max combining attempts
k_harq_soft_buffer_size	Max soft buffer size for Chase Combining $(1024*8+6)*2 = 16396$ bits, rounded to 16400
k_harq_survivors_buffer_size	Survivors buffer size

Definition at line 166 of file [rx_harq.h](#).

```

00166 : uint16_t {
00167   k_harq_max_payload      = 1024, /**< Maximum payload size in bytes */
00168   k_harq_default_retries  = 3,    /**< Default maximum retries */
00169   k_harq_default_combines = 3,    /**< Default max combining attempts */
00170   k_harq_soft_buffer_size = 16400, /**< Max soft buffer size for Chase Combining
00171                                     * (1024*8+6)*2 = 16396 bits, rounded to 16400 */
00172   k_harq_survivors_buffer_size = k_harq_soft_buffer_size / 2U, /**< Survivors buffer size */
00173 } rx_harq_params_t;
  
```

5.1.3.2 rx_harq_state_t

```
enum rx_harq_state_t : uint8_t
```

HARQ state machine states.

Enumerator

k_harq_state_idle	Ready for new transmission
k_harq_state_waiting_ack	Waiting for ACK
k_harq_state_combining	Combining retransmissions
k_harq_state_error	Unrecoverable error

Definition at line 183 of file rx_harq.h.

```
00183      : uint8_t {
00184  k_harq_state_idle      = 0, /**< Ready for new transmission */
00185  k_harq_state_waiting_ack = 1, /**< Waiting for ACK */
00186  k_harq_state_combining  = 2, /**< Combining retransmissions */
00187  k_harq_state_error      = 3, /**< Unrecoverable error */
00188 } rx_harq_state_t;
```

5.1.4 Function Documentation

5.1.4.1 rx_chase_combiner_add()

```
rx_err_t rx_chase_combiner_add (
    rx_chase_combiner_t * combiner,
    const rx_soft_bit_t * soft_bits,
    uint32_t len)
```

Add soft bits from a transmission attempt.

Accumulates soft bits element-wise. First call sets expected length.

Parameters

in,out	combiner	Initialized combiner handle
in	soft_bits	Received soft bits
in	len	Number of soft bits

Returns

k_rx_ok on success
 k_rx_err_invalid_arg if any pointer is nullptr or len is 0
 k_rx_err_invalid_size if len doesn't match expected length
 k_rx_err_busy if max combines reached

Definition at line 187 of file [rx_harq.c](#).

```

00188 {
00189     if (combiner == nullptr || soft_bits == nullptr) {
00190         return k_rx_err_invalid_arg;
00191     }
00192
00193     if (combiner->initialized == k_harq_false) {
00194         return k_rx_err_invalid_state;
00195     }
00196
00197     if (len == 0) {
00198         return k_rx_err_invalid_arg;
00199     }
00200
00201     if (len > k_harq_soft_buffer_size) {
00202         return k_rx_err_invalid_size;
00203     }
00204
00205     if (combiner->count >= combiner->max_combines) {
00206         return k_rx_err_busy;
00207     }
00208
00209     /* First transmission sets the expected length */
00210     if (combiner->count == 0) {
00211         combiner->expected_len = len;
00212         /* Clear accumulator for new frame */
00213         for (uint32_t i = 0U; i < len; i++) {
00214             combiner->accumulated[i] = 0;
00215         }
00216     } else if (len != combiner->expected_len) {
00217         return k_rx_err_invalid_size;
00218     }
00219
00220     /*
00221      * Element-wise addition (accumulate).
00222      *
00223      * Loop bound verification (NASA Rule 2):
00224      * - len is validated: len > 0 (line 62-64) AND len <= k_harq_soft_buffer_size (line 66-68)
00225      * - Maximum iterations: k_harq_soft_buffer_size (16400)
00226      * - Array bounds: accumulated[i] and soft_bits[i] are safe for i < len
00227      */
00228     for (uint32_t i = 0; i < len; i++) {
00229         combiner->accumulated[i] = (int16_t)(combiner->accumulated[i] + (int16_t)soft_bits[i]);
00230     }
00231
00232     combiner->count++;
00233     return k_rx_ok;
00234 }

```

References [rx_chase_combiner_t::accumulated](#), [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::expected_len](#), [rx_chase_combiner_t::initialized](#), [k_harq_false](#), [k_harq_soft_buffer_size](#), and [rx_chase_combiner_t::max_combines](#).

Referenced by [rx_harq_decode\(\)](#).

Here is the caller graph for this function:



5.1.4.2 rx_chase_combiner_can_add()

```

bool rx_chase_combiner_can_add (
    const rx\_chase\_combiner\_t * combiner) [nodiscard]

```

Check if more transmissions can be combined.

Parameters

in	combiner	Pointer to combiner handle
----	----------	----------------------------

Returns

true if count < max_combines

Check if more transmissions can be combined.

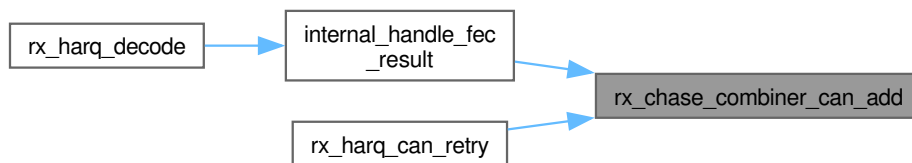
Definition at line 304 of file rx_harq.c.

```
00305 {
00306     if (combiner == nullptr) {
00307         return false;
00308     }
00309     if (combiner->initialized == k_harq_false) {
00310         return false;
00311     }
00312     return (bool)(combiner->count < combiner->max_combines);
00313 }
```

References [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::initialized](#), [k_harq_false](#), and [rx_chase_combiner_t::max_combines](#).

Referenced by [internal_handle_fec_result\(\)](#), and [rx_harq_can_retry\(\)](#).

Here is the caller graph for this function:



5.1.4.3 rx_chase_combiner_combined()

```
rx_err_t rx_chase_combiner_combined (
    const rx_chase_combiner_t * combiner,
    rx_soft_bit_t * output,
    uint32_t * len) [nodiscard]
```

Get combined soft bits for decoding.

Returns accumulated values clamped to SoftBit range [-127, +127].

Parameters

in	combiner	Initialized combiner handle
out	output	Output buffer for combined soft bits
out	len	Number of combined soft bits

Returns

`k_rx_ok` on success
`k_rx_err_invalid_arg` if any pointer is `nullptr`
`k_rx_err_invalid_state` if no soft bits have been added

Definition at line 236 of file `rx_harq.c`.

```

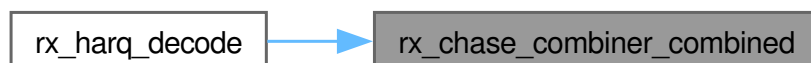
00239 {
00240     if (combiner == nullptr || output == nullptr || len == nullptr) {
00241         return k_rx_err_invalid_arg;
00242     }
00243     if (combiner->initialized == k_harq_false) {
00244         return k_rx_err_invalid_state;
00245     }
00246     const bool count_zero = (bool)(combiner->count == 0);
00247     const bool len_zero = (bool)(combiner->expected_len == 0);
00248     if ((bool)((int)count_zero | (int)len_zero)) {
00249         return k_rx_err_invalid_state;
00250     }
00251     /*
00252     * Clamp accumulated values to SoftBit range [-127, +127].
00253     * Loop bound verification (NASA Rule 2):
00254     * - expected_len is set only via rx_chase_combiner_add() with validated len
00255     * - expected_len > 0 (validated at line 103-105)
00256     * - expected_len <= k_harq_soft_buffer_size (validated when set in add)
00257     * - Maximum iterations: k_harq_soft_buffer_size (16400)
00258     * - Array bounds: accumulated[i] and output[i] are safe for i < expected_len
00259     */
00260     for (uint32_t i = 0; i < combiner->expected_len; i++) {
00261         const int16_t acc = combiner->accumulated[i];
00262         if (acc > k_soft_bit_max) {
00263             output[i] = k_soft_bit_max;
00264         } else if (acc < k_soft_bit_min) {
00265             output[i] = k_soft_bit_min;
00266         } else {
00267             output[i] = (rx_soft_bit_t)acc;
00268         }
00269     }
00270     *len = combiner->expected_len;
00271     return k_rx_ok;
00272 }

```

References `rx_chase_combiner_t::accumulated`, `rx_chase_combiner_t::count`, `rx_chase_combiner_t::expected_len`, `rx_chase_combiner_t::initialized`, and `k_harq_false`.

Referenced by `rx_harq_decode()`.

Here is the caller graph for this function:



5.1.4.4 rx_chase_combiner_count()

```

uint8_t rx_chase_combiner_count (
    const rx_chase_combiner_t * combiner)

```

Get number of combined transmissions.

Parameters

in	combiner	Pointer to combiner handle
----	----------	----------------------------

Returns

Number of transmissions combined

Get number of combined transmissions.

Definition at line 320 of file [rx_harq.c](#).

```
00321 {
00322     if (combiner == nullptr) {
00323         return 0;
00324     }
00325     return combiner->count;
00326 }
```

References [rx_chase_combiner_t::count](#).

5.1.4.5 rx_chase_combiner_deinit()

```
rx_err_t rx_chase_combiner_deinit (
    rx\_chase\_combiner\_t * combiner)    [nodiscard]
```

Deinitialize Chase Combiner.

Parameters

in,out	combiner	Pointer to combiner handle
--------	----------	----------------------------

Returns

k_rx_ok on success

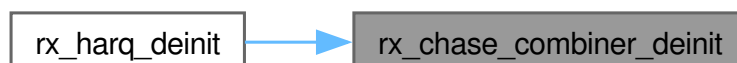
Definition at line 176 of file [rx_harq.c](#).

```
00177 {
00178     if (combiner == nullptr) {
00179         return k_rx_err_invalid_arg;
00180     }
00181     combiner->initialized = k_harq_false;
00182     return k_rx_ok;
00183 }
00184 }
```

References [rx_chase_combiner_t::initialized](#), and [k_harq_false](#).

Referenced by [rx_harq_deinit\(\)](#).

Here is the caller graph for this function:



5.1.4.6 rx_chase_combiner_init()

```
rx_err_t rx_chase_combiner_init (
    rx_chase_combiner_t * combiner,
    uint8_t max_combines)  [nodiscard]
```

Initialize Chase Combiner.

Parameters

out	combiner	Pointer to combiner handle
in	max_combines	Maximum number of combining attempts (0 = default)

Returns

k_rx_ok on success, k_rx_err_invalid_arg if combiner is nullptr

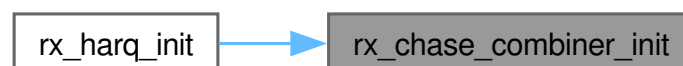
Definition at line 153 of file rx_harq.c.

```
00154 {
00155     if (combiner == nullptr) {
00156         return k_rx_err_invalid_arg;
00157     }
00158     /* Clear accumulator */
00159     for (uint32_t i = 0U; i < k_harq_soft_buffer_size; i++) {
00160         combiner->accumulated[i] = 0;
00161     }
00162     combiner->expected_len = 0;
00163     combiner->count = 0;
00164     if (max_combines > k_harq_zero_combines) {
00165         combiner->max_combines = max_combines;
00166     } else {
00167         combiner->max_combines = k_harq_default_combines;
00168     }
00169     combiner->initialized = k_harq_true;
00170     return k_rx_ok;
00171 }
00172
00173
00174 }
```

References [rx_chase_combiner_t::accumulated](#), [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::expected_len](#), [rx_chase_combiner_t::initialized](#), [k_harq_default_combines](#), [k_harq_soft_buffer_size](#), [k_harq_true](#), [k_harq_zero_combines](#), and [rx_chase_combiner_t::max_combines](#).

Referenced by [rx_harq_init\(\)](#).

Here is the caller graph for this function:



5.1.4.7 rx_chase_combiner_reset()

```
rx_err_t rx_chase_combiner_reset (
    rx_chase_combiner_t * combiner) [nodiscard]
```

Reset combiner for new frame.

Clears accumulated buffer and count.

Parameters

in,out	combiner	Pointer to combiner handle
--------	----------	----------------------------

Returns

k_rx_ok on success
 k_rx_err_invalid_arg if combiner is nullptr
 k_rx_err_invalid_state if combiner is not initialized

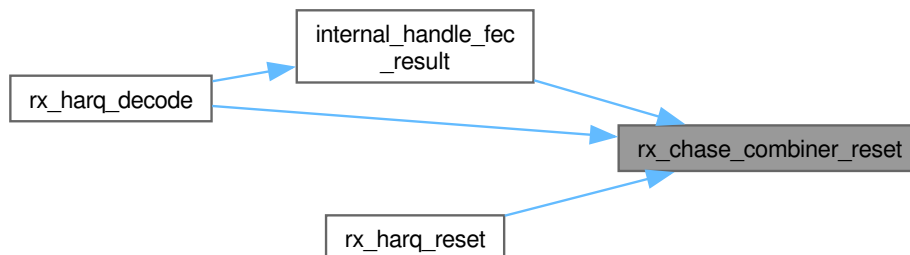
Definition at line 279 of file rx_harq.c.

```
00280 {
00281     if (combiner == nullptr) {
00282         return k_rx_err_invalid_arg;
00283     }
00284
00285     if (combiner->initialized == k_harq_false) {
00286         return k_rx_err_invalid_state;
00287     }
00288
00289     /* Clear the entire accumulator to prevent stale data */
00290     for (uint32_t i = 0U; i < k_harq_soft_buffer_size; i++) {
00291         combiner->accumulated[i] = 0;
00292     }
00293     combiner->count = 0;
00294     combiner->expected_len = 0;
00295
00296     return k_rx_ok;
00297 }
```

References [rx_chase_combiner_t::accumulated](#), [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::expected_len](#), [rx_chase_combiner_t::initialized](#), [k_harq_false](#), and [k_harq_soft_buffer_size](#).

Referenced by [internal_handle_fec_result\(\)](#), [rx_harq_decode\(\)](#), and [rx_harq_reset\(\)](#).

Here is the caller graph for this function:



5.1.4.8 rx_harq_can_retry()

```
bool rx_harq_can_retry (
    const rx_harq_handle_t * harq)  [nodiscard]
```

Check if more retries are available.

Parameters

in	harq	Pointer to HARQ handle
----	------	------------------------

Returns

true if retry_count < max_retries

Check if more retries are available.

Definition at line 666 of file rx_harq.c.

```
00667 {
00668     if (harq == nullptr) {
00669         return false;
00670     }
00671     if (harq->initialized == k_harq_false) {
00672         return false;
00673     }
00674     const bool retries_left = (bool)(harq->retry_count < harq->max_retries);
00675     const bool can_add      = rx_chase_combiner_can_add(&harq->combiner);
00676     return (bool)((int)retries_left & (int)can_add);
00677 }
```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [rx_harq_handle_t::max_retries](#), [rx_harq_handle_t::retry_count](#), and [rx_chase_combiner_can_add\(\)](#).

Here is the call graph for this function:



5.1.4.9 rx_harq_decode()

```
rx_err_t rx_harq_decode (
    rx_harq_handle_t * harq,
    const rx_harq_decode_params_t * params,
    uint8_t * output,
    uint32_t * output_len)  [nodiscard]
```

Decode soft bits with combining and FEC.

Adds soft bits to combiner, attempts decode. On failure, keeps accumulating.

Parameters

in	harq	HARQ handle
----	------	-------------

in	params	Decode parameters
out	output	Decoded output buffer
out	output_len	Actual decoded length

Returns

k_rx_ok on successful decode
 k_rx_err_protocol_error if decode failed (more retries possible)
 k_rx_err_invalid_arg if any pointer is nullptr
 k_rx_err_invalid_state if not initialized

Definition at line 584 of file [rx_harq.c](#).

```

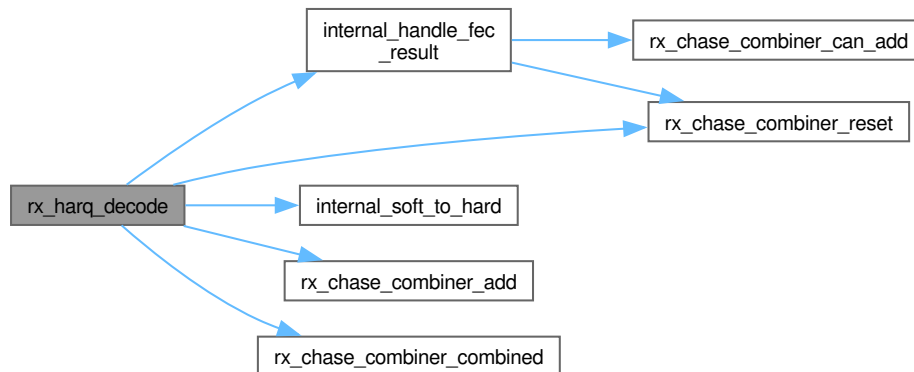
00588 {
00589     const bool harq_null      = (bool)(harq == nullptr);
00590     const bool params_null    = (bool)(params == nullptr);
00591     const bool output_null    = (bool)(output == nullptr);
00592     const bool output_len_null = (bool)(output_len == nullptr);
00593     if ((bool)((int)harq_null | (int)params_null | (int)output_null | (int)output_len_null)) {
00594         return k_rx_err_invalid_arg;
00595     }
00596     if (harq->initialized == k_harq_false) {
00597         return k_rx_err_invalid_state;
00598     }
00599     if (params->soft_bits == nullptr || params->soft_len == 0) {
00600         return k_rx_err_invalid_arg;
00601     }
00602
00603     /* Add soft bits to combiner */
00604     rx_err_t err = rx_chase_combiner_add(&harq->combiner, params->soft_bits, params->soft_len);
00605     const bool not_ok = (bool)(err != k_rx_ok);
00606     const bool not_busy = (bool)(err != k_rx_err_busy);
00607     if ((bool)((int)not_ok & (int)not_busy)) {
00608         return err;
00609     }
00610
00611     /* Get combined soft bits into handle's buffer (thread-safe) */
00612     uint32_t combined_len = 0U;
00613     (void)rx_chase_combiner_combined(&harq->combiner, harq->decode_buffer, &combined_len);
00614     /* Post-condition: combiner_combined only fails if count==0 or expected_len==0,
00615      * but combiner_add just succeeded (count>=1, expected_len>0) */
00616
00617     /* Non-FEC path: direct soft-to-hard conversion */
00618     if (harq->fec_enabled == k_harq_false) {
00619         err = internal_soft_to_hard(harq->decode_buffer,
00620                                     combined_len,
00621                                     params->expected_output_len,
00622                                     output,
00623                                     output_len);
00624         if (err != k_rx_ok) {
00625             return err;
00626         }
00627         harq->retry_count++;
00628         /* Ignore return - combiner is already validated via harq->initialized */
00629         (void)rx_chase_combiner_reset(&harq->combiner);
00630         harq->state = k_harq_state_idle;
00631         return k_rx_ok;
00632     }
00633
00634     /* FEC path: decode combined soft bits */
00635     const rx_fec_decode_soft_params_t decode_params = {
00636         .soft_bits      = harq->decode_buffer,
00637         .soft_len       = combined_len,
00638         .expected_output_len = params->expected_output_len,
00639         .output         = output,
00640         .output_len     = output_len,
00641     };
00642
00643     err = rx_fec_decode_soft(&harq->decoder, &decode_params);
00644
00645     return internal_handle_fec_result(harq, err);
00646 }

```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::decode_buffer](#), [rx_harq_handle_t::decoder](#), [rx_harq_decode_params_t::expected_output_len](#), [rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#),

[internal_handle_fec_result\(\)](#), [internal_soft_to_hard\(\)](#), [k_harq_false](#), [k_harq_state_idle](#), [rx_harq_handle_t::retry_c](#), [rx_chase_combiner_add\(\)](#), [rx_chase_combiner_combined\(\)](#), [rx_chase_combiner_reset\(\)](#), [rx_harq_decode_params_t](#), [rx_harq_decode_params_t::soft_len](#), and [rx_harq_handle_t::state](#).

Here is the call graph for this function:



5.1.4.10 rx_harq_deinit()

```
rx_err_t rx_harq_deinit (
    rx_harq_handle_t * harq) [nodiscard]
```

Deinitialize HARQ handle.

Parameters

in,out	harq	Pointer to HARQ handle
--------	------	------------------------

Returns

k_rx_ok on success

Deinitialize HARQ handle.

Definition at line 386 of file [rx_harq.c](#).

```

00387 {
00388     if (harq == nullptr) {
00389         return k_rx_err_invalid_arg;
00390     }
00391
00392     if (harq->fec_enabled == k_harq_true) {
00393         (void)(rx_fec_encoder_deinit(&harq->encoder));
00394         /* Post-condition: deinit only fails on nullptr; &harq->encoder is never null */
00395         (void)(rx_fec_decoder_deinit(&harq->decoder));
00396         /* Post-condition: deinit only fails on nullptr; &harq->decoder is never null */
00397     }
00398
00399     (void)(rx_chase_combiner_deinit(&harq->combiner));
00400     /* Post-condition: deinit only fails on nullptr; &harq->combiner is never null */
00401
00402     harq->initialized = k_harq_false;
00403     return k_rx_ok;
00404 }
```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::decoder](#), [rx_harq_handle_t::encoder](#), [rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [k_harq_true](#), and [rx_chase_combiner_deinit\(\)](#).

Here is the call graph for this function:



5.1.4.11 rx_harq_encode()

```
rx_err_t rx_harq_encode (
    const rx_harq_handle_t * harq,
    const uint8_t * payload,
    uint32_t payload_len,
    uint8_t * output,
    uint32_t output_size,
    uint32_t * output_len) [nodiscard]
```

Encode payload with FEC (if enabled).

Parameters

in	harq	HARQ handle
in	payload	Input payload
in	payload_len	Payload length
out	output	Output buffer (must be large enough for encoded data)
in	output_size	Size of output buffer in bytes
out	output_len	Actual output length

Returns

k_rx_ok on success
k_rx_err_invalid_arg if any pointer is nullptr
k_rx_err_invalid_size if output buffer too small
k_rx_err_invalid_state if not initialized

Definition at line 444 of file rx_harq.c.

```
00450 {
00451     if (harq == nullptr || payload == nullptr || output == nullptr || output_len == nullptr) {
00452         return k_rx_err_invalid_arg;
00453     }
00454
00455     if (harq->initialized == k_harq_false) {
00456         return k_rx_err_invalid_state;
00457     }
00458
00459     if (payload_len == 0 || payload_len > k_harq_max_payload) {
00460         return k_rx_err_invalid_size;
00461     }
00462
00463     /* If FEC is disabled, just copy payload */
00464     if (harq->fec_enabled == k_harq_false) {
00465         /* Validate output buffer can hold payload */
00466         if (output_size < payload_len) {
00467             return k_rx_err_invalid_size;
00468         }
00469         for (uint32_t i = 0U; i < payload_len; i++) {
00470             output[i] = payload[i];
00471         }
00472         *output_len = payload_len;
00473         return k_rx_ok;
00474     }
00475
00476     /*
00477     * FEC encode: Rate 1/2 convolutional code doubles output size.
00478     * Required buffer: (payload_len * 8 + 6) * 2 bits = (payload_len * 8 + 6) / 4 bytes
00479     * Simplified worst case: payload_len * 2 + 2 bytes
00480     */
00481     const uint32_t min_output_size = (payload_len * k_harq_fec_rate_multiplier) + k_harq_tail_bytes;
00482     if (output_size < min_output_size) {
00483         return k_rx_err_invalid_size;
00484     }
}
```

```

00485
00486  /* FEC encode */
00487  return rx_fec_encode(&harq->encoder, payload, payload_len, output, output_len);
00488 }

```

References [rx_harq_handle_t::encoder](#), [rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [k_harq_fec_rate_multiplier](#), [k_harq_max_payload](#), and [k_harq_tail_bytes](#).

5.1.4.12 rx_harq_get_retry_count()

```

uint8_t rx_harq_get_retry_count (
    const rx_harq_handle_t * harq)

```

Get current retry count.

Parameters

in	harq	Pointer to HARQ handle
----	------	------------------------

Returns

Current retry count

Get current retry count.

Definition at line [653](#) of file [rx_harq.c](#).

```

00654 {
00655     if (harq == nullptr) {
00656         return 0;
00657     }
00658     return harq->retry_count;
00659 }

```

References [rx_harq_handle_t::retry_count](#).

5.1.4.13 rx_harq_get_state()

```

rx_harq_state_t rx_harq_get_state (
    const rx_harq_handle_t * harq)

```

Get current HARQ state.

Parameters

in	harq	Pointer to HARQ handle
----	------	------------------------

Returns

Current HARQ state

Definition at line [411](#) of file [rx_harq.c](#).

```

00412 {
00413     if (harq == nullptr) {
00414         return k_harq_state_error;
00415     }
00416     return harq->state;
00417 }

```

References [k_harq_state_error](#), and [rx_harq_handle_t::state](#).

5.1.4.14 rx_harq_init()

```
rx_err_t rx_harq_init (
    rx_harq_handle_t * harq,
    const rx_harq_config_t * config) [nodiscard]
```

Initialize HARQ handle.

Parameters

out	harq	Pointer to HARQ handle
in	config	Configuration (NULL for defaults)

Returns

k_rx_ok on success
k_rx_err_invalid_arg if harq is nullptr

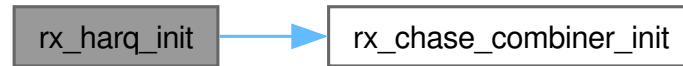
Definition at line 333 of file rx_harq.c.

```
00334 {
00335     if (harq == nullptr) {
00336         return k_rx_err_invalid_arg;
00337     }
00338
00339     /* Initialize state */
00340     harq->state = k_harq_state_idle;
00341     harq->tx_sequence = 0;
00342     harq->rx_sequence = 0;
00343     harq->retry_count = 0;
00344
00345     /* Apply configuration */
00346     if (config != nullptr) {
00347         if (config->max_retries > k_harq_zero_retries) {
00348             harq->max_retries = config->max_retries;
00349         } else {
00350             harq->max_retries = k_harq_default_retries;
00351         }
00352         harq->fec_enabled = config->fec_enabled;
00353     } else {
00354         harq->max_retries = k_harq_default_retries;
00355         harq->fec_enabled = k_harq_true; /* FEC enabled by default */
00356     }
00357
00358     /* Initialize Chase Combiner */
00359     const bool has_combines =
00360         (bool)(config != nullptr && (int)(config->max_combines > k_harq_zero_combines));
00361     const uint8_t max_combines = (int)has_combines ? config->max_combines : k_harq_default_combines;
00362     (void)(rx_chase_combiner_init(&harq->combiner, max_combines));
00363     /* Post-condition: combiner_init only fails on nullptr; &harq->combiner is never null */
00364
00365     /* Initialize FEC encoder if enabled */
00366     if (harq->fec_enabled == k_harq_true) {
00367         (void)(rx_fec_encoder_init(&harq->encoder));
00368         /* Post-condition: encoder_init only fails on nullptr; &harq->encoder is never null */
00369
00370         /* Initialize FEC decoder with survivors buffer */
00371         (void)(rx_fec_decoder_init(&harq->decoder,
00372             harq->decoder_survivors,
00373             k_harq_survivors_buffer_size));
00374         /* Post-condition: decoder_init only fails on nullptr or zero size; both are impossible here */
00375     }
00376
00377     harq->initialized = k_harq_true;
00378     return k_rx_ok;
00379 }
```

References rx_harq_handle_t::combiner, rx_harq_handle_t::decoder, rx_harq_handle_t::decoder_survivors, rx_harq_handle_t::encoder, rx_harq_config_t::fec_enabled, rx_harq_handle_t::fec_enabled, rx_harq_handle_t::initialized, k_harq_default_combines, k_harq_default_retries, k_harq_state_idle, k_harq_survivors_buffer_size, k_harq_true, k_harq_zero_combines, k_harq_zero_retries, rx_harq_config_t::max_c

[rx_harq_config_t::max_retries](#), [rx_harq_handle_t::max_retries](#), [rx_harq_handle_t::retry_count](#), [rx_chase_combiner_init\(\)](#), [rx_harq_handle_t::rx_sequence](#), [rx_harq_handle_t::state](#), and [rx_harq_handle_t::tx_seq](#)

Here is the call graph for this function:



5.1.4.15 rx_harq_reset()

```
rx_err_t rx_harq_reset (
    rx\_harq\_handle\_t * harq) [nodiscard]
```

Reset HARQ for new transaction.

Resets state to idle and clears combiner.

Parameters

in,out	harq	Pointer to HARQ handle
--------	------	------------------------

Returns

[k_rx_ok](#) on success
[k_rx_err_invalid_arg](#) if harq is nullptr
[k_rx_err_invalid_state](#) if harq is not initialized

Reset HARQ for new transaction.

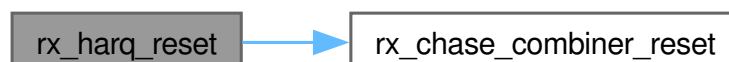
Definition at line 424 of file [rx_harq.c](#).

```

00425 {
00426     if (harq == nullptr) {
00427         return k_rx_err_invalid_arg;
00428     }
00429
00430     if (harq->initialized == k_harq_false) {
00431         return k_rx_err_invalid_state;
00432     }
00433
00434     harq->state = k_harq_state_idle;
00435     harq->retry_count = 0;
00436
00437     (void)(rx_chase_combiner_reset(&harq->combiner));
00438     /* Post-condition: combiner_reset only fails on nullptr or uninitialized;
00439      * &harq->combiner is always non-null and initialized after rx_harq_init() */
00440
00441     return k_rx_ok;
00442 }
```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [k_harq_state_idle](#), [rx_harq_handle_t::retry_count](#), [rx_chase_combiner_reset\(\)](#), and [rx_harq_handle_t::state](#).

Here is the call graph for this function:



5.2 rx_harq.h

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file rx_harq.h
00003  * @brief Hybrid Automatic Repeat Request (HARQ) Protocol for RX72N
00004  *
00005  * @details
00006  * Implements **Chase Combining HARQ Type I** for reliable communication over
00007  * lossy channels. Combines FEC (Forward Error Correction) with ARQ (Automatic
00008  * Repeat Request) to achieve both error correction and detection.
00009  *
00010  * ## HARQ Protocol Overview
00011  *
00012  * HARQ merges FEC and ARQ:
00013  * - **FEC**: Corrects random bit errors via convolutional coding
00014  * - **ARQ**: Requests retransmission when FEC fails
00015  * - **Chase Combining**: Accumulates soft bits from multiple transmissions
00016  *
00017  * ## Chase Combining Theory
00018  *
00019  * When a transmission fails to decode, instead of discarding it, the receiver:
00020  * 1. **Stores** soft bits from failed attempt
00021  * 2. **Requests** retransmission (NACK)
00022  * 3. **Adds** new soft bits to accumulated values (element-wise)
00023  * 4. **Decodes** combined soft bits (higher SNR -> better chance)
00024  *
00025  * ### Mathematical Model
00026  *
00027  * For @f$ n @f$ transmissions of the same data, the combined soft bit is:
00028  * @f[
00029  * s_{\text{combined}}[i] = \sum_{k=1}^n s_k[i]
00030  * @f]
00031  *
00032  * This provides **diversity gain**: independent noise samples are averaged out.
00033  *
00034  * ### SNR Improvement
00035  *
00036  * Assuming independent noise across transmissions, SNR improves as:
00037  * @f[
00038  * \text{SNR}_{\text{combined}} \approx n \cdot \text{SNR}_{\text{single}}
00039  * @f]
00040  *
00041  * In dB: @f$ 10 \log_{10}(n) @f$ gain (e.g., 3 dB for 2 transmissions, 4.8 dB for 3).
00042  *
00043  * ## Protocol Sequence
00044  *
00045  * @msc
00046  * Transmitter, Channel, Receiver;
00047  *
00048  * ... [label="Transmission 1"];
00049  * Transmitter box Transmitter [label="Encode with FEC\n100 bytes -> 202 bytes"];
00050  * Transmitter => Channel [label="Soft bits (1st attempt)"];
00051  * Channel box Channel [label="Add noise\nBER = 10^-3"];
00052  * Channel => Receiver [label="Noisy soft bits"];
00053  * Receiver box Receiver [label="Decode attempt\nFAILED"];
00054  * Receiver box Receiver [label="Store soft bits\nin combiner"];
00055  * Receiver => Transmitter [label="NACK (request retry)"];
00056  *
00057  * ... [label="Transmission 2"];
00058  * Transmitter => Channel [label="Soft bits (2nd attempt)"];
00059  * Channel box Channel [label="Independent noise\nBER = 10^-3"];
00060  * Channel => Receiver [label="Noisy soft bits"];
00061  * Receiver box Receiver [label="Add to combiner\nCombine attempts 1+2"];
00062  * Receiver box Receiver [label="Decode combined\nSUCCESS (3 dB gain)"];
00063  * Receiver => Transmitter [label="ACK (success)"];
00064  * Transmitter box Transmitter [label="Move to next frame"];
00065  * @endmsc
00066  *
00067  * ## HARQ State Machine
00068  *
00069  * @startuml
00070  * [*] --> Idle
00071  * Idle --> WaitingAck : tx_encode()
00072  * WaitingAck --> Idle : rx_ACK()
00073  * WaitingAck --> Combining : rx_NACK()
00074  * Combining --> Idle : decode_success()
00075  * Combining --> Error : max_retries_reached()
00076  * Error --> Idle : reset()
00077  * @enduml
00078  *
00079  * **State Descriptions:**
00080  * - **Idle**: Ready for new transmission, no pending frames
00081  * - **WaitingAck**: Waiting for ACK/NACK from receiver
00082  * - **Combining**: Accumulating retransmissions, attempting decode

```

```

00083 * - **Error**: Unrecoverable error (exceeded retry limit)
00084 *
00085 * **Transition Guards**:
00086 * - WaitingAck -> Combining: Only if retry_count < max_retries
00087 * - Combining -> Error: When retry_count >= max_retries AND decode still fails
00088 *
00089 * ## Performance Characteristics
00090 *
00091 * | Metric | Value |
00092 * |-----|-----|
00093 * | Throughput (1 tx) | 100 / 202 ~ 49.5% (FEC overhead) |
00094 * | Throughput (2 tx avg) | 100 / 404 ~ 24.8% (2x retransmissions) |
00095 * | Latency (1 tx) | ~1 ms encode + transmit |
00096 * | Latency (2 tx) | ~2-5 ms (includes RTT for NACK) |
00097 * | Memory (per handle) | ~82 KB (dominated by FEC decoder) |
00098 *
00099 * ## Features
00100 *
00101 * - [OK] **Chase Combining**: Soft bit accumulation (int16 to prevent overflow)
00102 * - [OK] **FEC Integration**: Optional K=7 rate 1/2 convolutional code
00103 * - [OK] **Configurable Retries**: 1-255 attempts (default: 3)
00104 * - [OK] **Static Allocation**: Zero malloc (NASA Rule 3)
00105 * - [OK] **Thread Safety**: Safe if handles not shared
00106 *
00107 * @par Hardware Requirements
00108 * | Resource | Usage |
00109 * |-----|-----|
00110 * | CPU | RX72N @ 240 MHz |
00111 * | RAM | ~82 KB per HARQ handle |
00112 * | Flash | ~8 KB (code size with FEC) |
00113 *
00114 * @par Module Dependencies
00115 * - [rx_fec.h](rx_fec_8h.html): FEC encoder/decoder
00116 * - [rx_err.h](rx_err_8h.html): Error codes
00117 *
00118 * @par NASA Power of 10 Compliance
00119 * - **Rule 1**: [OK] No recursion, goto, setjmp/longjmp
00120 * - **Rule 2**: [OK] All loops bounded (k_harq_soft_buffer_size)
00121 * - **Rule 3**: [OK] Zero dynamic allocation
00122 * - **Rule 4**: [OK] Functions <= 60 lines
00123 * - **Rule 5**: [OK] All functions have >=2 pre/postconditions
00124 *
00125 * @note **Thread Safety**: HARQ handles are NOT thread-safe. Caller must
00126 * provide synchronization if sharing handles between threads.
00127 *
00128 * @warning Soft bit accumulation uses int16 (range +/-32768). With max 3 combines
00129 * and soft bits +/-127, accumulator saturates at +/-381 (no overflow).
00130 *
00131 * @see star-gateway/internal/harq/ Go reference implementation
00132 * @see star-gateway/internal/fec/combiner.go Chase Combiner algorithm
00133 * @see docs/sections/01_nanopb_protocol.tex Protocol specification
00134 * @see "Hybrid ARQ Schemes" by S. Lin & D. Costello
00135 *
00136 * @author Locked, Inc.
00137 * @date 2026-01-27
00138 * @copyright Copyright (c) 2026 Locked Inc.
00139 * SPDX-License-Identifier: MIT
00140 *
00141 * @since Version 1.0.0
00142 * @version 1.0.0
00143 *
00144 * @test test_rx_harq.c Unit tests for HARQ protocol
00145 */
00146
00147 #pragma once
00148
00149 #include <stdint.h>
00150
00151 #include "rx_err.h"
00152 #include "rx_fec.h"
00153
00154 #ifdef __cplusplus
00155 extern "C" {
00156 #endif
00157
00158 /*
=====
00159 * HARQ Parameters
00160 *
=====
00161 */
00162
00163 /**
00164 * @brief HARQ protocol parameters
00165 */
00166 typedef enum : uint16_t {
00167 k_harq_max_payload = 1024, /**< Maximum payload size in bytes */

```

```

00168 k_harq_default_retries    = 3,    /**< Default maximum retries */
00169 k_harq_default_combines   = 3,    /**< Default max combining attempts */
00170 k_harq_soft_buffer_size   = 16400, /**< Max soft buffer size for Chase Combining
00171                               * (1024*8+6)*2 = 16396 bits, rounded to 16400 */
00172 k_harq_survivors_buffer_size = k_harq_soft_buffer_size / 2U, /**< Survivors buffer size */
00173 } rx_harq_params_t;
00174
00175 /*
=====
00176 * HARQ State Machine
00177 *
=====
00178 */
00179 /**
00180 * @brief HARQ state machine states
00181 */
00182 typedef enum : uint8_t {
00183     k_harq_state_idle      = 0, /**< Ready for new transmission */
00184     k_harq_state_waiting_ack = 1, /**< Waiting for ACK */
00185     k_harq_state_combining = 2, /**< Combining retransmissions */
00186     k_harq_state_error     = 3, /**< Unrecoverable error */
00187 } rx_harq_state_t;
00188
00189
00190 /*
=====
00191 * Chase Combiner
00192 *
=====
00193 */
00194 /**
00195 * @brief Chase Combiner for soft bit accumulation
00196 *
00197 * Stores soft bits from failed transmission attempts and combines them
00198 * element-wise (addition) before decoding. Uses int16 accumulators to
00199 * prevent overflow.
00200 */
00201 typedef struct {
00202     int16_t accumulated[k_harq_soft_buffer_size]; /**< Accumulated soft bits */
00203     uint32_t expected_len; /**< Expected soft bits len */
00204     uint8_t count; /**< Transmissions combined */
00205     uint8_t max_combines; /**< Max combining attempts */
00206     uint8_t initialized; /**< Non-zero if initialized */
00207 } rx_chase_combiner_t;
00208
00209 /**
00210 * @brief Initialize Chase Combiner
00211 *
00212 * @param[out] combiner Pointer to combiner handle
00213 * @param[in] max_combines Maximum number of combining attempts (0 = default)
00214 * @return k_rx_ok on success, k_rx_err_invalid_arg if combiner is nullptr
00215 */
00216 [[nodiscard]] rx_err_t rx_chase_combiner_init(rx_chase_combiner_t* combiner, uint8_t max_combines);
00217
00218 /**
00219 * @brief Deinitialize Chase Combiner
00220 *
00221 * @param[in,out] combiner Pointer to combiner handle
00222 * @return k_rx_ok on success
00223 */
00224 [[nodiscard]] rx_err_t rx_chase_combiner_deinit(rx_chase_combiner_t* combiner);
00225
00226 /**
00227 * @brief Add soft bits from a transmission attempt
00228 *
00229 * Accumulates soft bits element-wise. First call sets expected length.
00230 */
00231 /**
00232 * @param[in,out] combiner Initialized combiner handle
00233 * @param[in] soft_bits Received soft bits
00234 * @param[in] len Number of soft bits
00235 *
00236 * @return k_rx_ok on success
00237 * @return k_rx_err_invalid_arg if any pointer is nullptr or len is 0
00238 * @return k_rx_err_invalid_size if len doesn't match expected length
00239 * @return k_rx_err_busy if max combines reached
00240 */
00241 rx_err_t rx_chase_combiner_add(rx_chase_combiner_t* combiner, const rx_soft_bit_t* soft_bits, uint32_t len);
00242
00243 /**
00244 * @brief Get combined soft bits for decoding
00245 *
00246 * Returns accumulated values clamped to SoftBit range [-127, +127].
00247 *
00248 * @param[in] combiner Initialized combiner handle
00249 * @param[out] output Output buffer for combined soft bits
00250 */

```

```

00251 * @param[out] len Number of combined soft bits
00252 *
00253 * @return k_rx_ok on success
00254 * @return k_rx_err_invalid_arg if any pointer is nullptr
00255 * @return k_rx_err_invalid_state if no soft bits have been added
00256 */
00257 [[nodiscard]] rx_err_t rx_chase_combiner_combined(const rx_chase_combiner_t* combiner,
00258                                                    rx_soft_bit_t* output,
00259                                                    uint32_t* len);
00260
00261 /**
00262 * @brief Reset combiner for new frame
00263 *
00264 * Clears accumulated buffer and count.
00265 *
00266 * @param[in,out] combiner Pointer to combiner handle
00267 *
00268 * @return k_rx_ok on success
00269 * @return k_rx_err_invalid_arg if combiner is nullptr
00270 * @return k_rx_err_invalid_state if combiner is not initialized
00271 */
00272 [[nodiscard]] rx_err_t rx_chase_combiner_reset(rx_chase_combiner_t* combiner);
00273
00274 /**
00275 * @brief Check if more transmissions can be combined
00276 *
00277 * @param[in] combiner Pointer to combiner handle
00278 * @return true if count < max_combines
00279 */
00280 [[nodiscard]] bool rx_chase_combiner_can_add(const rx_chase_combiner_t* combiner);
00281
00282 /**
00283 * @brief Get number of combined transmissions
00284 *
00285 * @param[in] combiner Pointer to combiner handle
00286 * @return Number of transmissions combined
00287 */
00288 uint8_t rx_chase_combiner_count(const rx_chase_combiner_t* combiner);
00289
00290 /*
=====
00291 * HARQ Handle
00292 *
=====
00293 */
00294
00295 /**
00296 * @brief HARQ protocol handle
00297 *
00298 * Contains all state for HARQ operation including FEC encoder/decoder
00299 * and Chase Combiner.
00300 */
00301 typedef struct {
00302     rx_harq_state_t state; /**< Current HARQ state */
00303     uint16_t tx_sequence; /**< TX sequence number */
00304     uint16_t rx_sequence; /**< RX sequence number */
00305     uint8_t retry_count; /**< Current retry count */
00306     uint8_t max_retries; /**< Maximum retries allowed */
00307     rx_chase_combiner_t combiner; /**< Chase Combiner */
00308     rx_fec_encoder_t encoder; /**< FEC encoder */
00309     rx_fec_decoder_t decoder; /**< FEC decoder */
00310     uint64_t decoder_survivors[k_harq_survivors_buffer_size]; /**< Decoder buf */
00311     rx_soft_bit_t decode_buffer[k_harq_soft_buffer_size]; /**< Combined soft bits buf */
00312     uint8_t fec_enabled; /**< Non-zero if FEC is enabled */
00313     uint8_t initialized; /**< Non-zero if initialized */
00314 } rx_harq_handle_t;
00315
00316 /**
00317 * @brief HARQ configuration
00318 */
00319 typedef struct {
00320     uint8_t max_retries; /**< Maximum transmission attempts (0 = default) */
00321     uint8_t fec_enabled; /**< Non-zero to enable FEC encoding/decoding */
00322     uint8_t max_combines; /**< Maximum combining attempts (0 = default) */
00323 } rx_harq_config_t;
00324
00325 /**
00326 * @brief HARQ decode parameters
00327 */
00328 typedef struct {
00329     const rx_soft_bit_t* soft_bits; /**< Received soft bits */
00330     uint32_t soft_len; /**< Number of soft bits */
00331     uint32_t expected_output_len; /**< Expected decoded length */
00332 } rx_harq_decode_params_t;
00333
00334 /**
00335 * @brief Initialize HARQ handle

```

```

00336 *
00337 * @param[out] harq Pointer to HARQ handle
00338 * @param[in] config Configuration (NULL for defaults)
00339 *
00340 * @return k_rx_ok on success
00341 * @return k_rx_err_invalid_arg if harq is nullptr
00342 */
00343 [[nodiscard]] rx_err_t rx_harq_init(rx_harq_handle_t* harq, const rx_harq_config_t* config);
00344
00345 /**
00346 * @brief Deinitialize HARQ handle
00347 *
00348 * @param[in,out] harq Pointer to HARQ handle
00349 * @return k_rx_ok on success
00350 */
00351 [[nodiscard]] rx_err_t rx_harq_deinit(rx_harq_handle_t* harq);
00352
00353 /**
00354 * @brief Get current HARQ state
00355 *
00356 * @param[in] harq Pointer to HARQ handle
00357 * @return Current HARQ state
00358 */
00359 rx_harq_state_t rx_harq_get_state(const rx_harq_handle_t* harq);
00360
00361 /**
00362 * @brief Reset HARQ for new transaction
00363 *
00364 * Resets state to idle and clears combiner.
00365 *
00366 * @param[in,out] harq Pointer to HARQ handle
00367 *
00368 * @return k_rx_ok on success
00369 * @return k_rx_err_invalid_arg if harq is nullptr
00370 * @return k_rx_err_invalid_state if harq is not initialized
00371 */
00372 [[nodiscard]] rx_err_t rx_harq_reset(rx_harq_handle_t* harq);
00373
00374 /**
00375 * @brief Encode payload with FEC (if enabled)
00376 *
00377 * @param[in] harq HARQ handle
00378 * @param[in] payload Input payload
00379 * @param[in] payload_len Payload length
00380 * @param[out] output Output buffer (must be large enough for encoded data)
00381 * @param[in] output_size Size of output buffer in bytes
00382 * @param[out] output_len Actual output length
00383 *
00384 * @return k_rx_ok on success
00385 * @return k_rx_err_invalid_arg if any pointer is nullptr
00386 * @return k_rx_err_invalid_size if output buffer too small
00387 * @return k_rx_err_invalid_state if not initialized
00388 */
00389 [[nodiscard]] rx_err_t rx_harq_encode(const rx_harq_handle_t* harq,
00390                                     const uint8_t* payload,
00391                                     uint32_t payload_len,
00392                                     uint8_t* output,
00393                                     uint32_t output_size,
00394                                     uint32_t* output_len);
00395
00396 /**
00397 * @brief Decode soft bits with combining and FEC
00398 *
00399 * Adds soft bits to combiner, attempts decode. On failure, keeps accumulating.
00400 *
00401 * @param[in] harq HARQ handle
00402 * @param[in] params Decode parameters
00403 * @param[out] output Decoded output buffer
00404 * @param[out] output_len Actual decoded length
00405 *
00406 * @return k_rx_ok on successful decode
00407 * @return k_rx_err_protocol_error if decode failed (more retries possible)
00408 * @return k_rx_err_invalid_arg if any pointer is nullptr
00409 * @return k_rx_err_invalid_state if not initialized
00410 */
00411 [[nodiscard]] rx_err_t rx_harq_decode(rx_harq_handle_t* harq,
00412                                     const rx_harq_decode_params_t* params,
00413                                     uint8_t* output,
00414                                     uint32_t* output_len);
00415
00416 /**
00417 * @brief Get current retry count
00418 *
00419 * @param[in] harq Pointer to HARQ handle
00420 * @return Current retry count
00421 */
00422 uint8_t rx_harq_get_retry_count(const rx_harq_handle_t* harq);

```

```

00423
00424 /**
00425  * @brief Check if more retries are available
00426  *
00427  * @param[in] harq Pointer to HARQ handle
00428  * @return true if retry_count < max_retries
00429  */
00430 [[nodiscard]] bool rx_harq_can_retry(const rx_harq_handle_t* harq);
00431
00432 #ifdef __cplusplus
00433 }
00434 #endif

```

5.3 libs/rx_harq/src/rx_harq.c File Reference

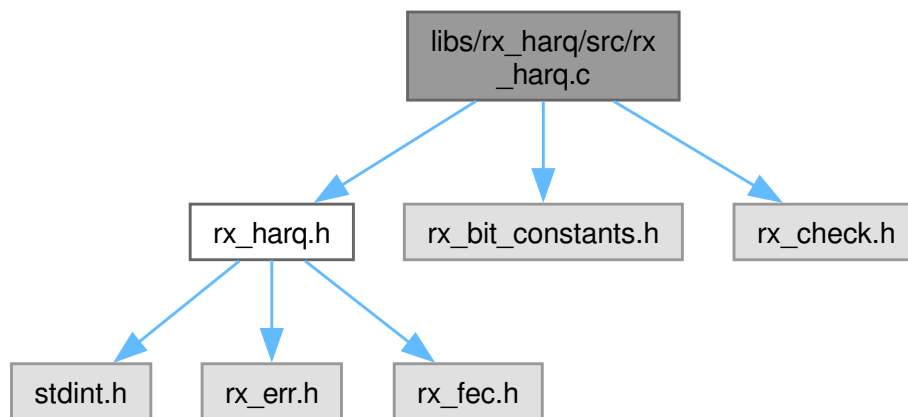
Hybrid Automatic Repeat Request (HARQ) Protocol Implementation.

```

#include "rx_harq.h"
#include "rx_bit_constants.h"
#include "rx_check.h"

```

Include dependency graph for rx_harq.c:



Enumerations

- enum `harq_fec_constants_t` : `uint16_t` { `k_harq_fec_rate_multiplier` = 2 , `k_harq_tail_bytes` = 2 }
- HARQ FEC size constants.
- enum `harq_cfg_sentinel_t` : `uint8_t` { `k_harq_zero_combines` = 0 , `k_harq_zero_retries` = 0 }
- HARQ config validation sentinels – zero values for "not configured, use default".
- enum `harq_bool_t` : `uint8_t` { `k_harq_false` = 0U , `k_harq_true` = 1U }
- enum `bit_constants_t` : `uint8_t` { `k_msb_position` = `k_rx_bits_per_byte` - 1U , `k_rounding_adjustment` = `k_rx_bits_per_byte` - 1U , `k_soft_bit_zero_thresh` = 0 }
- Bit position constants for soft-to-hard conversion.

Functions

- `rx_err_t rx_chase_combiner_init (rx_chase_combiner_t *combiner, const uint8_t max_combines)`
Initialize Chase Combiner.
- `rx_err_t rx_chase_combiner_deinit (rx_chase_combiner_t *combiner)`
Deinitialize Chase Combiner.
- `rx_err_t rx_chase_combiner_add (rx_chase_combiner_t *combiner, const rx_soft_bit_t *soft_bits, uint32_t len)`
Add soft bits from a transmission attempt.
- `rx_err_t rx_chase_combiner_combined (const rx_chase_combiner_t *combiner, rx_soft_bit_t *output, uint32_t *len)`
Get combined soft bits for decoding.
- `rx_err_t rx_chase_combiner_reset (rx_chase_combiner_t *combiner)`
Reset combiner for new frame.
- `bool rx_chase_combiner_can_add (const rx_chase_combiner_t *combiner)`
Check if combiner can accept more transmissions.
- `uint8_t rx_chase_combiner_count (const rx_chase_combiner_t *combiner)`
Get number of transmissions combined so far.
- `rx_err_t rx_harq_init (rx_harq_handle_t *harq, const rx_harq_config_t *config)`
Initialize HARQ handle.
- `rx_err_t rx_harq_deinit (rx_harq_handle_t *harq)`
Deinitialize HARQ handle and release resources.
- `rx_harq_state_t rx_harq_get_state (const rx_harq_handle_t *harq)`
Get current HARQ state.
- `rx_err_t rx_harq_reset (rx_harq_handle_t *harq)`
Reset HARQ state for new transmission.
- `rx_err_t rx_harq_encode (const rx_harq_handle_t *harq, const uint8_t *payload, const uint32_t payload_len, uint8_t *output, const uint32_t output_size, uint32_t *output_len)`
Encode payload with FEC (if enabled).
- `static rx_err_t internal_soft_to_hard (const rx_soft_bit_t *soft_bits, const uint32_t soft_bit_count, const uint32_t max_output_bytes, uint8_t *output, uint32_t *output_len)`
Convert combined soft bits to hard bits (non-FEC path).
- `static rx_err_t internal_handle_fec_result (rx_harq_handle_t *harq, const rx_err_t result)`
Handle FEC decode result and update HARQ state.
- `rx_err_t rx_harq_decode (rx_harq_handle_t *harq, const rx_harq_decode_params_t *params, uint8_t *output, uint32_t *output_len)`
Decode soft bits with combining and FEC.
- `uint8_t rx_harq_get_retry_count (const rx_harq_handle_t *harq)`
Get number of retries attempted.
- `bool rx_harq_can_retry (const rx_harq_handle_t *harq)`
Check if more retries are allowed.

5.3.1 Detailed Description

Hybrid Automatic Repeat Request (HARQ) Protocol Implementation.

Implements Chase Combining HARQ Type I combining FEC with soft bit accumulation across retransmissions. This is a C port of star-gateway/internal/harq/ and fec/combiner.go for bit-exact compatibility between RX72N and RPi5.

5.3.1.1 Implementation Architecture

5.3.1.1.1 Three-Layer Design

1. Chase Combiner ([rx_chase_combiner_t](#))
 - Accumulates soft bits element-wise across transmissions
 - Uses int16 accumulators (prevents overflow with ≤ 255 combines)
 - Clamps output to $[-127, +127]$ range before decoding
2. FEC Codec ([rx_fec_encoder_t](#), [rx_fec_decoder_t](#))
 - Optional layer for error correction
 - $K=7$, rate $1/2$ convolutional code
 - Integrated seamlessly with combiner
3. HARQ State Machine ([rx_harq_handle_t](#))
 - Manages retransmission protocol
 - Tracks retry count and sequence numbers
 - Coordinates encoder, decoder, and combiner

5.3.1.2 Chase Combining Algorithm

For n transmissions, accumulate soft bits:

$$\text{acc}[i] = \sum_{k=1}^n s_k[i]$$

Then clamp to valid range:

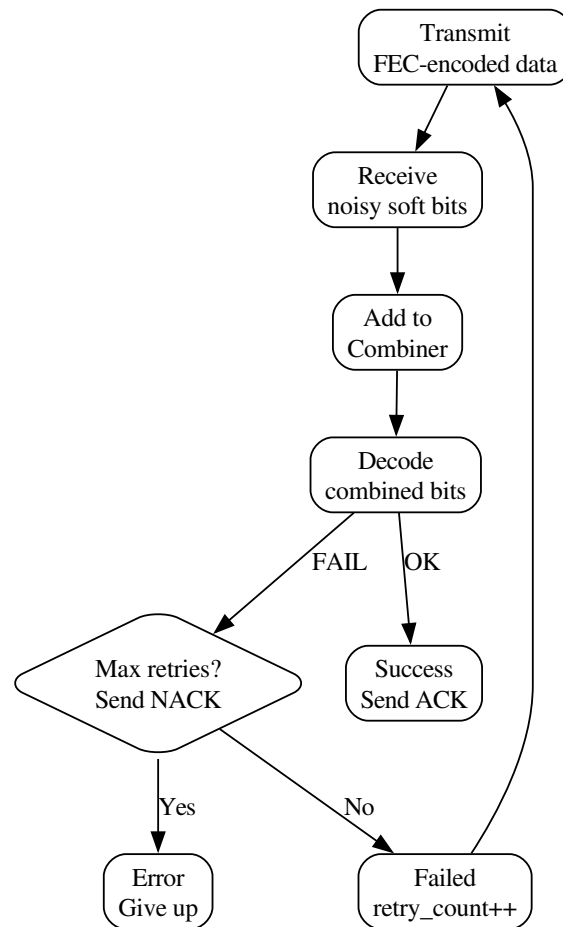
$$s_{\text{combined}}[i] = \begin{cases} +127 & \text{if } \text{acc}[i] > 127 \\ -127 & \text{if } \text{acc}[i] < -127 \\ \text{acc}[i] & \text{otherwise} \end{cases}$$

5.3.1.3 Overflow Prevention

Accumulators use int16 (range ± 32768):

- Max combines: 255
- Max soft bit: ± 127
- Max accumulator value: $\pm(255 \times 127) = \pm 32385$
- Safe: $32385 < 32768$ (no overflow with default `max_combines=3`)

5.3.1.4 Protocol Flow



Module Dependencies

- [rx_harq.h](#): Public API
- [rx_fec.h](#): FEC codec
- [rx_bit_constants.h](#): Bit manipulation

NASA Power of 10 Compliance

- Rule 1: [OK] No recursion, goto, setjmp/longjmp
- Rule 2: [OK] Loops bounded by `k_harq_soft_buffer_size`
- Rule 3: [OK] Zero dynamic allocation
- Rule 4: [OK] Functions ≤ 60 lines
- Rule 5: [OK] Extensive validation checks

Author

Locked, Inc.

Date

2026-01-27

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Since

Version 1.0.0

Version

1.0.0

Definition in file [rx_harq.c](#).

5.3.2 Enumeration Type Documentation

5.3.2.1 bit_constants_t

enum [bit_constants_t](#) : uint8_t

Bit position constants for soft-to-hard conversion.

Enumerator

k_msb_position	Most significant bit position
k_rounding_adjustment	Ceiling division: $(n + 7) / 8$
k_soft_bit_zero_thresh	Threshold for soft-to-hard: ≥ 0 is bit 1

Definition at line 498 of file [rx_harq.c](#).

```
00498      : uint8_t {
00499  k_msb_position      = k_rx_bits_per_byte - 1U, /**< Most significant bit position */
00500  k_rounding_adjustment = k_rx_bits_per_byte - 1U, /**< Ceiling division: (n + 7) / 8 */
00501  k_soft_bit_zero_thresh = 0, /**< Threshold for soft-to-hard:  $\geq 0$  is bit 1 */
00502 } bit_constants_t;
```

5.3.2.2 harq_bool_t

enum [harq_bool_t](#) : uint8_t

Enumerator

k_harq_false	
k_harq_true	

Definition at line 143 of file [rx_harq.c](#).

```
00143      : uint8_t {
00144  k_harq_false = 0U,
00145  k_harq_true  = 1U,
00146 } harq_bool_t;
```

5.3.2.3 harq_cfg_sentinel_t

enum [harq_cfg_sentinel_t](#) : uint8_t

HARQ config validation sentinels – zero values for "not configured, use default".

Used to detect unconfigured (zero) values for max_combines and max_retries fields in HARQ configuration, to fall back to project defaults.

Invariant

k_harq_zero_combines and k_harq_zero_retries equal zero, signifying "use default config" – these sentinels must never be changed to non-zero values.

```
// Checking sentinels when validating config fields:
const rx_harq_config_t config = { .max_combines = 0, .max_retries = 0 };
// max_combines == k_harq_zero_combines -> use k_harq_default_combines
// max_retries == k_harq_zero_retries -> use k_harq_default_retries
uint8_t combines = (config.max_combines > k_harq_zero_combines)
    ? config.max_combines : k_harq_default_combines;
uint8_t retries = (config.max_retries > k_harq_zero_retries)
    ? config.max_retries : k_harq_default_retries;
```

See also

[rx_harq_config_t](#) HARQ configuration structure containing max_combines/max_retries

[rx_harq_init\(\)](#) Applies these sentinels during initialization

Since

Version 1.0.0

Enumerator

k_harq_zero_combines	Sentinel: no max_combines configured -> fall back to default
k_harq_zero_retries	Sentinel: no max_retries configured -> fall back to default

Definition at line 138 of file [rx_harq.c](#).

```
00138     : uint8_t {
00139     k_harq_zero_combines = 0, /**< Sentinel: no max_combines configured -> fall back to default */
00140     k_harq_zero_retries = 0, /**< Sentinel: no max_retries configured -> fall back to default */
00141 } harq_cfg_sentinel_t;
```

5.3.2.4 harq_fec_constants_t

enum [harq_fec_constants_t](#) : uint16_t

HARQ FEC size constants.

Enumerator

k_harq_fec_rate_multiplier	Rate 1/2: encoded length multiplier
k_harq_tail_bytes	Tail overhead in bytes

Definition at line 107 of file [rx_harq.c](#).

```
00107     : uint16_t {
00108     k_harq_fec_rate_multiplier = 2, /**< Rate 1/2: encoded length multiplier */
00109     k_harq_tail_bytes         = 2, /**< Tail overhead in bytes */
00110 } harq_fec_constants_t;
```

5.3.3 Function Documentation

5.3.3.1 internal_handle_fec_result()

```
rx_err_t internal_handle_fec_result (
    rx_harq_handle_t * harq,
    const rx_err_t result) [static]
```

Handle FEC decode result and update HARQ state.

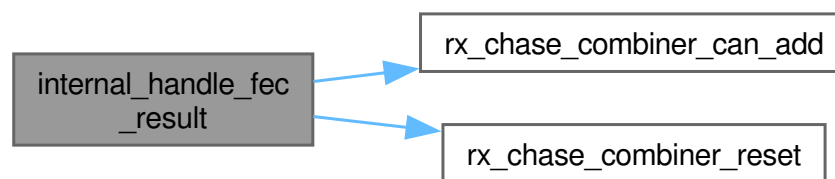
Definition at line 552 of file [rx_harq.c](#).

```
00553 {
00554     /* Pre-condition: only called from rx_harq_decode where harq is already validated */
00555
00556     harq->retry_count++;
00557
00558     if (result == k_rx_ok) {
00559         /* Ignore return - combiner is already validated via harq->initialized */
00560         (void)rx_chase_combiner_reset(&harq->combiner);
00561         harq->state = k_harq_state_idle;
00562         return k_rx_ok;
00563     }
00564
00565     /* Decode failed - check if we can retry */
00566     const bool can_add_more = rx_chase_combiner_can_add(&harq->combiner);
00567     const bool retries_remain = (bool)(harq->retry_count < harq->max_retries);
00568     if ((bool)((int)can_add_more & (int)retries_remain)) {
00569         harq->state = k_harq_state_combining;
00570         return k_rx_err_protocol_error; /* Need more retransmissions */
00571     }
00572
00573     /* Max retries reached */
00574     (void)rx_chase_combiner_reset(&harq->combiner);
00575     harq->state = k_harq_state_error;
00576     return result;
00577 }
```

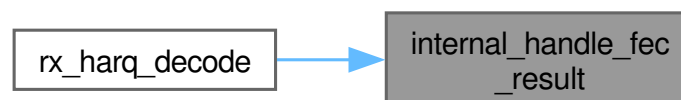
References [rx_harq_handle_t::combiner](#), [k_harq_state_combining](#), [k_harq_state_error](#), [k_harq_state_idle](#), [rx_harq_handle_t::max_retries](#), [rx_harq_handle_t::retry_count](#), [rx_chase_combiner_can_add\(\)](#), [rx_chase_combiner_reset\(\)](#), and [rx_harq_handle_t::state](#).

Referenced by [rx_harq_decode\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.3.3.2 internal_soft_to_hard()

```
rx_err_t internal_soft_to_hard (
    const rx_soft_bit_t * soft_bits,
    const uint32_t soft_bit_count,
    const uint32_t max_output_bytes,
    uint8_t * output,
    uint32_t * output_len) [static]
```

Convert combined soft bits to hard bits (non-FEC path).

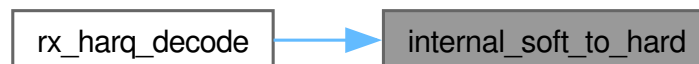
Definition at line 509 of file [rx_harq.c](#).

```
00514 {
00515     /* Pre-conditions: pointers are always valid from rx_harq_decode (harq->decode_buffer,
00516      * caller-validated output, caller-validated output_len) */
00517     if (max_output_bytes == 0) {
00518         return k_rx_err_invalid_arg;
00519     }
00520
00521     /* Calculate output bytes with ceiling division */
00522     uint32_t out_bytes = (soft_bit_count + k_rounding_adjustment) / k_rx_bits_per_byte;
00523     if (out_bytes > max_output_bytes) {
00524         out_bytes = max_output_bytes;
00525     }
00526     for (uint32_t i = 0U; i < out_bytes; i++) {
00527         output[i] = 0U;
00528     }
00529
00530     /*
00531      * Loop bound: i < soft_len is validated by prior check that soft_bits != nullptr.
00532      * Loop terminates when i >= soft_len OR when byte index exceeds out_bytes.
00533      * Maximum iterations: min(soft_len, out_bytes * k_rx_bits_per_byte).
00534      */
00535     for (uint32_t i = 0; i < soft_bit_count && (i / k_rx_bits_per_byte) < out_bytes; i++) {
00536         if (soft_bits[i] >= k_soft_bit_zero_thresh) {
00537             const uint32_t byte_idx = i / k_rx_bits_per_byte;
00538             const uint32_t bit_pos = k_msb_position - (i % k_rx_bits_per_byte);
00539             output[byte_idx] |= (uint8_t)(1U << bit_pos);
00540         }
00541     }
00542
00543     *output_len = out_bytes;
00544     return k_rx_ok;
00545 }
```

References [k_msb_position](#), [k_rounding_adjustment](#), and [k_soft_bit_zero_thresh](#).

Referenced by [rx_harq_decode\(\)](#).

Here is the caller graph for this function:



5.3.3.3 rx_chase_combiner_add()

```
rx_err_t rx_chase_combiner_add (
    rx_chase_combiner_t * combiner,
    const rx_soft_bit_t * soft_bits,
    uint32_t len)
```

Add soft bits from a transmission attempt.

Accumulates soft bits element-wise. First call sets expected length.

Parameters

in,out	combiner	Initialized combiner handle
in	soft_bits	Received soft bits
in	len	Number of soft bits

Returns

k_rx_ok on success
 k_rx_err_invalid_arg if any pointer is nullptr or len is 0
 k_rx_err_invalid_size if len doesn't match expected length
 k_rx_err_busy if max combines reached

Definition at line 187 of file [rx_harq.c](#).

```

00188 {
00189     if (combiner == nullptr || soft_bits == nullptr) {
00190         return k_rx_err_invalid_arg;
00191     }
00192
00193     if (combiner->initialized == k_harq_false) {
00194         return k_rx_err_invalid_state;
00195     }
00196
00197     if (len == 0) {
00198         return k_rx_err_invalid_arg;
00199     }
00200
00201     if (len > k_harq_soft_buffer_size) {
00202         return k_rx_err_invalid_size;
00203     }
00204
00205     if (combiner->count >= combiner->max_combines) {
00206         return k_rx_err_busy;
00207     }
00208
00209     /* First transmission sets the expected length */
00210     if (combiner->count == 0) {
00211         combiner->expected_len = len;
00212         /* Clear accumulator for new frame */
00213         for (uint32_t i = 0U; i < len; i++) {
00214             combiner->accumulated[i] = 0;
00215         }
00216     } else if (len != combiner->expected_len) {
00217         return k_rx_err_invalid_size;
00218     }
00219
00220     /*
00221      * Element-wise addition (accumulate).
00222      *
00223      * Loop bound verification (NASA Rule 2):
00224      * - len is validated: len > 0 (line 62-64) AND len <= k_harq_soft_buffer_size (line 66-68)
00225      * - Maximum iterations: k_harq_soft_buffer_size (16400)
00226      * - Array bounds: accumulated[i] and soft_bits[i] are safe for i < len
00227      */
00228     for (uint32_t i = 0; i < len; i++) {
00229         combiner->accumulated[i] = (int16_t)(combiner->accumulated[i] + (int16_t)soft_bits[i]);
00230     }
00231
00232     combiner->count++;
00233     return k_rx_ok;
00234 }

```

References [rx_chase_combiner_t::accumulated](#), [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::expected_len](#), [rx_chase_combiner_t::initialized](#), [k_harq_false](#), [k_harq_soft_buffer_size](#), and [rx_chase_combiner_t::max_combines](#).

Referenced by [rx_harq_decode\(\)](#).

Here is the caller graph for this function:



5.3.3.4 rx_chase_combiner_can_add()

```
bool rx_chase_combiner_can_add (
    const rx_chase_combiner_t * combiner) [nodiscard]
```

Check if combiner can accept more transmissions.

Check if more transmissions can be combined.

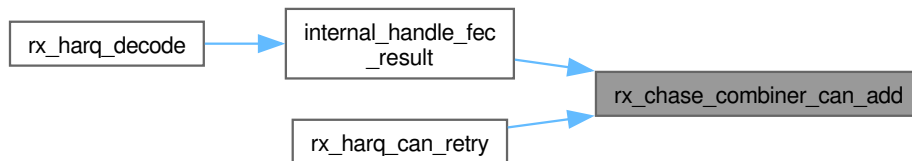
Definition at line 304 of file rx_harq.c.

```
00305 {
00306     if (combiner == nullptr) {
00307         return false;
00308     }
00309     if (combiner->initialized == k_harq_false) {
00310         return false;
00311     }
00312     return (bool)(combiner->count < combiner->max_combines);
00313 }
```

References [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::initialized](#), [k_harq_false](#), and [rx_chase_combiner_t::max_combines](#).

Referenced by [internal_handle_fec_result\(\)](#), and [rx_harq_can_retry\(\)](#).

Here is the caller graph for this function:



5.3.3.5 rx_chase_combiner_combined()

```
rx_err_t rx_chase_combiner_combined (
    const rx_chase_combiner_t * combiner,
    rx_soft_bit_t * output,
    uint32_t * len) [nodiscard]
```

Get combined soft bits for decoding.

Returns accumulated values clamped to SoftBit range [-127, +127].

Parameters

in	combiner	Initialized combiner handle
out	output	Output buffer for combined soft bits
out	len	Number of combined soft bits

Returns

`k_rx_ok` on success
`k_rx_err_invalid_arg` if any pointer is nullptr
`k_rx_err_invalid_state` if no soft bits have been added

Definition at line 236 of file `rx_harq.c`.

```

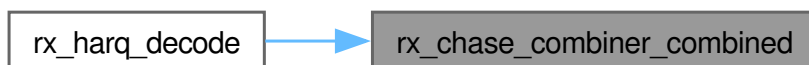
00239 {
00240     if (combiner == nullptr || output == nullptr || len == nullptr) {
00241         return k_rx_err_invalid_arg;
00242     }
00243
00244     if (combiner->initialized == k_harq_false) {
00245         return k_rx_err_invalid_state;
00246     }
00247
00248     const bool count_zero = (bool)(combiner->count == 0);
00249     const bool len_zero = (bool)(combiner->expected_len == 0);
00250     if ((bool)((int)count_zero | (int)len_zero)) {
00251         return k_rx_err_invalid_state;
00252     }
00253
00254     /*
00255     * Clamp accumulated values to SoftBit range [-127, +127].
00256     *
00257     * Loop bound verification (NASA Rule 2):
00258     * - expected_len is set only via rx_chase_combiner_add() with validated len
00259     * - expected_len > 0 (validated at line 103-105)
00260     * - expected_len <= k_harq_soft_buffer_size (validated when set in add)
00261     * - Maximum iterations: k_harq_soft_buffer_size (16400)
00262     * - Array bounds: accumulated[i] and output[i] are safe for i < expected_len
00263     */
00264     for (uint32_t i = 0; i < combiner->expected_len; i++) {
00265         const int16_t acc = combiner->accumulated[i];
00266         if (acc > k_soft_bit_max) {
00267             output[i] = k_soft_bit_max;
00268         } else if (acc < k_soft_bit_min) {
00269             output[i] = k_soft_bit_min;
00270         } else {
00271             output[i] = (rx_soft_bit_t)acc;
00272         }
00273     }
00274
00275     *len = combiner->expected_len;
00276     return k_rx_ok;
00277 }

```

References `rx_chase_combiner_t::accumulated`, `rx_chase_combiner_t::count`, `rx_chase_combiner_t::expected_len`, `rx_chase_combiner_t::initialized`, and `k_harq_false`.

Referenced by `rx_harq_decode()`.

Here is the caller graph for this function:



5.3.3.6 rx_chase_combiner_count()

```

uint8_t rx_chase_combiner_count (
    const rx_chase_combiner_t * combiner)

```

Get number of transmissions combined so far.

Get number of combined transmissions.

Definition at line 320 of file `rx_harq.c`.

```

00321 {
00322     if (combiner == nullptr) {
00323         return 0;
00324     }
00325     return combiner->count;
00326 }

```

References `rx_chase_combiner_t::count`.

5.3.3.7 rx_chase_combiner_deinit()

```
rx_err_t rx_chase_combiner_deinit (
    rx_chase_combiner_t * combiner)  [nodiscard]
```

Deinitialize Chase Combiner.

Parameters

in,out	combiner	Pointer to combiner handle
--------	----------	----------------------------

Returns

k_rx_ok on success

Definition at line 176 of file rx_harq.c.

```
00177 {
00178     if (combiner == nullptr) {
00179         return k_rx_err_invalid_arg;
00180     }
00181
00182     combiner->initialized = k_harq_false;
00183     return k_rx_ok;
00184 }
```

References [rx_chase_combiner_t::initialized](#), and [k_harq_false](#).

Referenced by [rx_harq_deinit\(\)](#).

Here is the caller graph for this function:



5.3.3.8 rx_chase_combiner_init()

```
rx_err_t rx_chase_combiner_init (
    rx_chase_combiner_t * combiner,
    uint8_t max_combines)  [nodiscard]
```

Initialize Chase Combiner.

Parameters

out	combiner	Pointer to combiner handle
in	max_combines	Maximum number of combining attempts (0 = default)

Returns

k_rx_ok on success, k_rx_err_invalid_arg if combiner is nullptr

Definition at line 153 of file rx_harq.c.

```

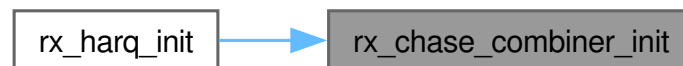
00154 {
00155     if (combiner == nullptr) {
00156         return k_rx_err_invalid_arg;
00157     }
00158
00159     /* Clear accumulator */
00160     for (uint32_t i = 0U; i < k_harq_soft_buffer_size; i++) {
00161         combiner->accumulated[i] = 0;
00162     }
00163
00164     combiner->expected_len = 0;
00165     combiner->count = 0;
00166     if (max_combines > k_harq_zero_combines) {
00167         combiner->max_combines = max_combines;
00168     } else {
00169         combiner->max_combines = k_harq_default_combines;
00170     }
00171     combiner->initialized = k_harq_true;
00172
00173     return k_rx_ok;
00174 }

```

References [rx_chase_combiner_t::accumulated](#), [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::expected_len](#), [rx_chase_combiner_t::initialized](#), [k_harq_default_combines](#), [k_harq_soft_buffer_size](#), [k_harq_true](#), [k_harq_zero_combines](#), and [rx_chase_combiner_t::max_combines](#).

Referenced by [rx_harq_init\(\)](#).

Here is the caller graph for this function:



5.3.3.9 rx_chase_combiner_reset()

```

rx_err_t rx_chase_combiner_reset (
    rx\_chase\_combiner\_t * combiner) [nodiscard]

```

Reset combiner for new frame.

Clears accumulated buffer and count.

Parameters

in,out	combiner	Pointer to combiner handle
--------	----------	----------------------------

Returns

`k_rx_ok` on success
`k_rx_err_invalid_arg` if combiner is nullptr
`k_rx_err_invalid_state` if combiner is not initialized

Definition at line 279 of file `rx_harq.c`.

```

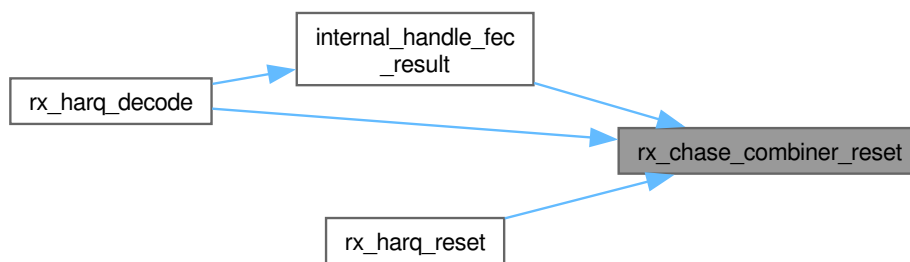
00280 {
00281     if (combiner == nullptr) {
00282         return k_rx_err_invalid_arg;
00283     }
00284
00285     if (combiner->initialized == k_harq_false) {
00286         return k_rx_err_invalid_state;
00287     }
00288
00289     /* Clear the entire accumulator to prevent stale data */
00290     for (uint32_t i = 0U; i < k_harq_soft_buffer_size; i++) {
00291         combiner->accumulated[i] = 0;
00292     }
00293     combiner->count = 0;
00294     combiner->expected_len = 0;
00295
00296     return k_rx_ok;
00297 }

```

References `rx_chase_combiner_t::accumulated`, `rx_chase_combiner_t::count`, `rx_chase_combiner_t::expected_len`, `rx_chase_combiner_t::initialized`, `k_harq_false`, and `k_harq_soft_buffer_size`.

Referenced by `internal_handle_fec_result()`, `rx_harq_decode()`, and `rx_harq_reset()`.

Here is the caller graph for this function:



5.3.3.10 rx_harq_can_retry()

```

bool rx_harq_can_retry (
    const rx_harq_handle_t * harq) [nodiscard]

```

Check if more retries are allowed.

Check if more retries are available.

Definition at line 666 of file `rx_harq.c`.

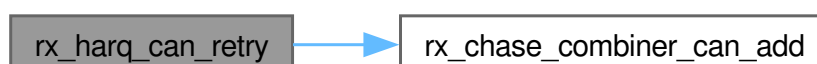
```

00667 {
00668     if (harq == nullptr) {
00669         return false;
00670     }
00671     if (harq->initialized == k_harq_false) {
00672         return false;
00673     }
00674     const bool retries_left = (bool)(harq->retry_count < harq->max_retries);
00675     const bool can_add = rx_chase_combiner_can_add(&harq->combiner);
00676     return (bool)((int)retries_left & (int)can_add);
00677 }

```

References `rx_harq_handle_t::combiner`, `rx_harq_handle_t::initialized`, `k_harq_false`, `rx_harq_handle_t::max_retries`, `rx_harq_handle_t::retry_count`, and `rx_chase_combiner_can_add()`.

Here is the call graph for this function:



5.3.3.11 rx_harq_decode()

```
rx_err_t rx_harq_decode (
    rx_harq_handle_t * harq,
    const rx_harq_decode_params_t * params,
    uint8_t * output,
    uint32_t * output_len) [nodiscard]
```

Decode soft bits with combining and FEC.

Adds soft bits to combiner, attempts decode. On failure, keeps accumulating.

Parameters

in	harq	HARQ handle
in	params	Decode parameters
out	output	Decoded output buffer
out	output_len	Actual decoded length

Returns

k_rx_ok on successful decode
k_rx_err_protocol_error if decode failed (more retries possible)
k_rx_err_invalid_arg if any pointer is nullptr
k_rx_err_invalid_state if not initialized

Definition at line 584 of file rx_harq.c.

```
00588 {
00589     const bool harq_null = (bool)(harq == nullptr);
00590     const bool params_null = (bool)(params == nullptr);
00591     const bool output_null = (bool)(output == nullptr);
00592     const bool output_len_null = (bool)(output_len == nullptr);
00593     if ((bool)((int)harq_null | (int)params_null | (int)output_null | (int)output_len_null)) {
00594         return k_rx_err_invalid_arg;
00595     }
00596     if (harq->initialized == k_harq_false) {
00597         return k_rx_err_invalid_state;
00598     }
00599     if (params->soft_bits == nullptr || params->soft_len == 0) {
00600         return k_rx_err_invalid_arg;
00601     }
00602     /* Add soft bits to combiner */
00603     rx_err_t err = rx_chase_combiner_add(&harq->combiner, params->soft_bits, params->soft_len);
00604     const bool not_ok = (bool)(err != k_rx_ok);
00605     const bool not_busy = (bool)(err != k_rx_err_busy);
00606     if ((bool)((int)not_ok & (int)not_busy)) {
00607         return err;
00608     }
00609     /* Get combined soft bits into handle's buffer (thread-safe) */
00610     uint32_t combined_len = 0U;
00611     (void)rx_chase_combiner_combined(&harq->combiner, harq->decode_buffer, &combined_len);
00612     /* Post-condition: combiner_combined only fails if count==0 or expected_len==0,
00613      * but combiner_add just succeeded (count>=1, expected_len>0) */
00614     /* Non-FEC path: direct soft-to-hard conversion */
00615     if (harq->fec_enabled == k_harq_false) {
00616         err = internal_soft_to_hard(harq->decode_buffer,
00617                                     combined_len,
00618                                     params->expected_output_len,
00619                                     output,
00620                                     output_len);
00621         if (err != k_rx_ok) {
00622             return err;
00623         }
00624     }
00625 }
```

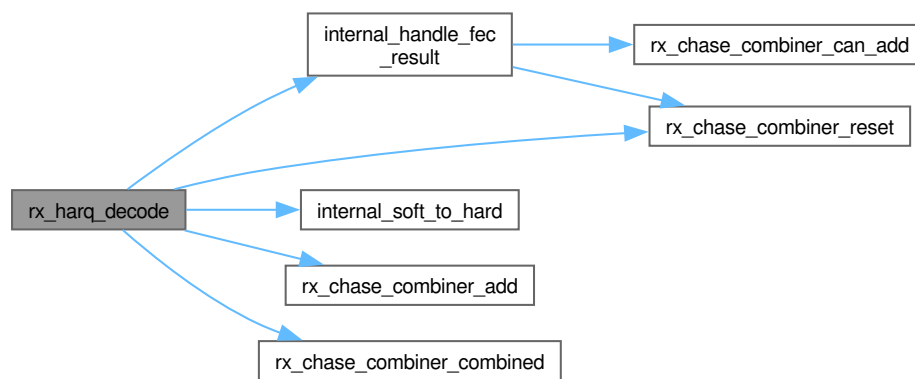
```

00627     harq->retry_count++;
00628     /* Ignore return - combiner is already validated via harq->initialized */
00629     (void)rx_chase_combiner_reset(&harq->combiner);
00630     harq->state = k_harq_state_idle;
00631     return k_rx_ok;
00632 }
00633
00634 /* FEC path: decode combined soft bits */
00635 const rx_fec_decode_soft_params_t decode_params = {
00636     .soft_bits      = harq->decode_buffer,
00637     .soft_len       = combined_len,
00638     .expected_output_len = params->expected_output_len,
00639     .output         = output,
00640     .output_len     = output_len,
00641 };
00642
00643 err = rx_fec_decode_soft(&harq->decoder, &decode_params);
00644
00645 return internal_handle_fec_result(harq, err);
00646 }

```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::decode_buffer](#), [rx_harq_handle_t::decoder](#), [rx_harq_decode_params_t::expected_output_len](#), [rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [internal_handle_fec_result\(\)](#), [internal_soft_to_hard\(\)](#), [k_harq_false](#), [k_harq_state_idle](#), [rx_harq_handle_t::retry_count](#), [rx_chase_combiner_add\(\)](#), [rx_chase_combiner_combined\(\)](#), [rx_chase_combiner_reset\(\)](#), [rx_harq_decode_params_t::soft_len](#), and [rx_harq_handle_t::state](#).

Here is the call graph for this function:



5.3.3.12 rx_harq_deinit()

```

rx_err_t rx_harq_deinit (
    rx_harq_handle_t * harq) [nodiscard]

```

Deinitialize HARQ handle and release resources.

Deinitialize HARQ handle.

Definition at line 386 of file [rx_harq.c](#).

```

00387 {
00388     if (harq == nullptr) {
00389         return k_rx_err_invalid_arg;
00390     }
00391
00392     if (harq->fec_enabled == k_harq_true) {
00393         (void)rx_fec_encoder_deinit(&harq->encoder);
00394         /* Post-condition: deinit only fails on nullptr; &harq->encoder is never null */
00395         (void)rx_fec_decoder_deinit(&harq->decoder);
00396         /* Post-condition: deinit only fails on nullptr; &harq->decoder is never null */
00397     }
00398
00399     (void)rx_chase_combiner_deinit(&harq->combiner);
00400     /* Post-condition: deinit only fails on nullptr; &harq->combiner is never null */

```

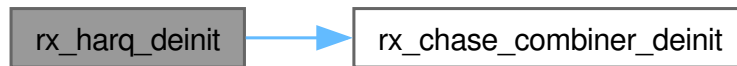
```

00401
00402 harq->initialized = k_harq_false;
00403 return k_rx_ok;
00404 }

```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::decoder](#), [rx_harq_handle_t::encoder](#), [rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [k_harq_true](#), and [rx_chase_combiner_deinit\(\)](#).

Here is the call graph for this function:



5.3.3.13 rx_harq_encode()

```

rx_err_t rx_harq_encode (
    const rx_harq_handle_t * harq,
    const uint8_t * payload,
    uint32_t payload_len,
    uint8_t * output,
    uint32_t output_size,
    uint32_t * output_len) [nodiscard]

```

Encode payload with FEC (if enabled).

Parameters

in	harq	HARQ handle
in	payload	Input payload
in	payload_len	Payload length
out	output	Output buffer (must be large enough for encoded data)
in	output_size	Size of output buffer in bytes
out	output_len	Actual output length

Returns

- k_rx_ok on success
- k_rx_err_invalid_arg if any pointer is nullptr
- k_rx_err_invalid_size if output buffer too small
- k_rx_err_invalid_state if not initialized

Definition at line 444 of file [rx_harq.c](#).

```

00450 {
00451     if (harq == nullptr || payload == nullptr || output == nullptr || output_len == nullptr) {
00452         return k_rx_err_invalid_arg;
00453     }
00454
00455     if (harq->initialized == k_harq_false) {
00456         return k_rx_err_invalid_state;
00457     }
00458
00459     if (payload_len == 0 || payload_len > k_harq_max_payload) {
00460         return k_rx_err_invalid_size;

```

```

00461 }
00462
00463 /* If FEC is disabled, just copy payload */
00464 if (harq->fec_enabled == k_harq_false) {
00465     /* Validate output buffer can hold payload */
00466     if (output_size < payload_len) {
00467         return k_rx_err_invalid_size;
00468     }
00469     for (uint32_t i = 0U; i < payload_len; i++) {
00470         output[i] = payload[i];
00471     }
00472     *output_len = payload_len;
00473     return k_rx_ok;
00474 }
00475
00476 /*
00477  * FEC encode: Rate 1/2 convolutional code doubles output size.
00478  * Required buffer: (payload_len * 8 + 6) * 2 bits = (payload_len * 8 + 6) / 4 bytes
00479  * Simplified worst case: payload_len * 2 + 2 bytes
00480  */
00481 const uint32_t min_output_size = (payload_len * k_harq_fec_rate_multiplier) + k_harq_tail_bytes;
00482 if (output_size < min_output_size) {
00483     return k_rx_err_invalid_size;
00484 }
00485
00486 /* FEC encode */
00487 return rx_fec_encode(&harq->encoder, payload, payload_len, output, output_len);
00488 }

```

References [rx_harq_handle_t::encoder](#), [rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [k_harq_fec_rate_multiplier](#), [k_harq_max_payload](#), and [k_harq_tail_bytes](#).

5.3.3.14 rx_harq_get_retry_count()

```

uint8_t rx_harq_get_retry_count (
    const rx_harq_handle_t * harq)

```

Get number of retries attempted.

Get current retry count.

Definition at line 653 of file [rx_harq.c](#).

```

00654 {
00655     if (harq == nullptr) {
00656         return 0;
00657     }
00658     return harq->retry_count;
00659 }

```

References [rx_harq_handle_t::retry_count](#).

5.3.3.15 rx_harq_get_state()

```

rx_harq_state_t rx_harq_get_state (
    const rx_harq_handle_t * harq)

```

Get current HARQ state.

Parameters

in	harq	Pointer to HARQ handle
----	------	------------------------

Returns

Current HARQ state

Definition at line 411 of file [rx_harq.c](#).

```
00412 {
00413     if (harq == nullptr) {
00414         return k_harq_state_error;
00415     }
00416     return harq->state;
00417 }
```

References [k_harq_state_error](#), and [rx_harq_handle_t::state](#).

5.3.3.16 rx_harq_init()

```
rx_err_t rx_harq_init (
    rx_harq_handle_t * harq,
    const rx_harq_config_t * config) [nodiscard]
```

Initialize HARQ handle.

Parameters

out	harq	Pointer to HARQ handle
in	config	Configuration (NULL for defaults)

Returns

k_rx_ok on success
k_rx_err_invalid_arg if harq is nullptr

Definition at line 333 of file [rx_harq.c](#).

```
00334 {
00335     if (harq == nullptr) {
00336         return k_rx_err_invalid_arg;
00337     }
00338
00339     /* Initialize state */
00340     harq->state = k_harq_state_idle;
00341     harq->tx_sequence = 0;
00342     harq->rx_sequence = 0;
00343     harq->retry_count = 0;
00344
00345     /* Apply configuration */
00346     if (config != nullptr) {
00347         if (config->max_retries > k_harq_zero_retries) {
00348             harq->max_retries = config->max_retries;
00349         } else {
00350             harq->max_retries = k_harq_default_retries;
00351         }
00352         harq->fec_enabled = config->fec_enabled;
00353     } else {
00354         harq->max_retries = k_harq_default_retries;
00355         harq->fec_enabled = k_harq_true; /* FEC enabled by default */
00356     }
00357
00358     /* Initialize Chase Combiner */
00359     const bool has_combines =
00360         (bool)(config != nullptr && (int)(config->max_combines > k_harq_zero_combines));
00361     const uint8_t max_combines = (int)has_combines ? config->max_combines : k_harq_default_combines;
00362     (void)(rx_chase_combiner_init(&harq->combiner, max_combines));
00363     /* Post-condition: combiner_init only fails on nullptr; &harq->combiner is never null */
00364
00365     /* Initialize FEC encoder if enabled */
00366     if (harq->fec_enabled == k_harq_true) {
```

```

00367 (void)(rx_fec_encoder_init(&harq->encoder));
00368 /* Post-condition: encoder_init only fails on nullptr; &harq->encoder is never null */
00369
00370 /* Initialize FEC decoder with survivors buffer */
00371 (void)(rx_fec_decoder_init(&harq->decoder,
00372                          harq->decoder_survivors,
00373                          k_harq_survivors_buffer_size));
00374 /* Post-condition: decoder_init only fails on nullptr or zero size; both are impossible here */
00375 }
00376
00377 harq->initialized = k_harq_true;
00378 return k_rx_ok;
00379 }

```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::decoder](#), [rx_harq_handle_t::decoder_survivors](#), [rx_harq_handle_t::encoder](#), [rx_harq_config_t::fec_enabled](#), [rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [k_harq_default_combines](#), [k_harq_default_retries](#), [k_harq_state_idle](#), [k_harq_survivors_buffer_size](#), [k_harq_true](#), [k_harq_zero_combines](#), [k_harq_zero_retries](#), [rx_harq_config_t::max_c](#), [rx_harq_config_t::max_retries](#), [rx_harq_handle_t::max_retries](#), [rx_harq_handle_t::retry_count](#), [rx_chase_combiner_init\(\)](#), [rx_harq_handle_t::rx_sequence](#), [rx_harq_handle_t::state](#), and [rx_harq_handle_t::tx_seq](#).

Here is the call graph for this function:



5.3.3.17 rx_harq_reset()

```

rx_err_t rx_harq_reset (
    rx_harq_handle_t * harq) [nodiscard]

```

Reset HARQ state for new transmission.

Reset HARQ for new transaction.

Definition at line 424 of file [rx_harq.c](#).

```

00425 {
00426     if (harq == nullptr) {
00427         return k_rx_err_invalid_arg;
00428     }
00429
00430     if (harq->initialized == k_harq_false) {
00431         return k_rx_err_invalid_state;
00432     }
00433
00434     harq->state = k_harq_state_idle;
00435     harq->retry_count = 0;
00436
00437     (void)(rx_chase_combiner_reset(&harq->combiner));
00438     /* Post-condition: combiner_reset only fails on nullptr or uninitialized;
00439      * &harq->combiner is always non-null and initialized after rx_harq_init() */
00440
00441     return k_rx_ok;
00442 }

```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [k_harq_state_idle](#), [rx_harq_handle_t::retry_count](#), [rx_chase_combiner_reset\(\)](#), and [rx_harq_handle_t::state](#).

Here is the call graph for this function:



5.4 rx_harq.c

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file rx_harq.c
00003  * @brief Hybrid Automatic Repeat Request (HARQ) Protocol Implementation
00004  *
00005  * @details
00006  * Implements Chase Combining HARQ Type I combining FEC with soft bit accumulation
00007  * across retransmissions. This is a C port of star-gateway/internal/harq/ and
00008  * fec/combiner.go for bit-exact compatibility between RX72N and RPi5.
00009  *
00010  * ## Implementation Architecture
00011  *
00012  * ### Three-Layer Design
00013  *
00014  * 1. **Chase Combiner** (rx_chase_combiner_t)
00015  *   - Accumulates soft bits element-wise across transmissions
00016  *   - Uses int16 accumulators (prevents overflow with <=255 combines)
00017  *   - Clamps output to [-127, +127] range before decoding
00018  *
00019  * 2. **FEC Codec** (rx_fec_encoder_t, rx_fec_decoder_t)
00020  *   - Optional layer for error correction
00021  *   - K=7, rate 1/2 convolutional code
00022  *   - Integrated seamlessly with combiner
00023  *
00024  * 3. **HARQ State Machine** (rx_harq_handle_t)
00025  *   - Manages retransmission protocol
00026  *   - Tracks retry count and sequence numbers
00027  *   - Coordinates encoder, decoder, and combiner
00028  *
00029  * ## Chase Combining Algorithm
00030  *
00031  * For @f$ n @f$ transmissions, accumulate soft bits:
00032  *
00033  * @f[
00034  *   \text{acc}[i] = \sum_{k=1}^n s_k[i]
00035  * @f]
00036  *
00037  * Then clamp to valid range:
00038  *
00039  * @f[
00040  *   s_{\text{combined}}[i] = \begin{cases}
00041  *     +127 & \text{if } \text{acc}[i] > 127 \\
00042  *     -127 & \text{if } \text{acc}[i] < -127 \\
00043  *     \text{acc}[i] & \text{otherwise}
00044  *   \end{cases}
00045  * @f]
00046  *
00047  * ## Overflow Prevention
00048  *
00049  * Accumulators use int16 (range +/-32768):
00050  * - Max combines: 255
00051  * - Max soft bit: +/-127
00052  * - Max accumulator value: +/- (255 x 127) = +/-32385
00053  * - **Safe**: 32385 < 32768 (no overflow with default max_combines=3)
00054  *
00055  * ## Protocol Flow
00056  *
00057  * @dot
00058  * digraph harq_flow {
00059  *   rankdir=TB;
00060  *   node [shape=box, style=rounded];
00061  *
00062  *   tx [label="Transmit\nFEC-encoded data"];
00063  *   rx [label="Receive\nnoisy soft bits"];
00064  *   add [label="Add to\nCombiner"];
00065  *   decode [label="Decode\ncombined bits"];
00066  *   success [label="Success\nSend ACK", fillcolor=green];
00067  *   retry [label="Failed\nretry_count++"];
00068  *   max [label="Max retries?\nSend NACK", shape=diamond];
00069  *   error [label="Error\nGive up", fillcolor=red];
00070  *
00071  *   tx -> rx -> add -> decode;
00072  *   decode -> success [label="OK"];
00073  *   decode -> max [label="FAIL"];
00074  *   max -> retry [label="No"];
00075  *   retry -> tx;
00076  *   max -> error [label="Yes"];
00077  * }
00078  * @enddot
00079  *
00080  * @par Module Dependencies
00081  * - [rx_harq.h](rx_harq_8h.html): Public API
00082  * - [rx_fec.h](rx_fec_8h.html): FEC codec

```

```

00083 * - [rx_bit_constants.h](rx_bit_constants_8h.html): Bit manipulation
00084 *
00085 * @par NASA Power of 10 Compliance
00086 * - **Rule 1**: [OK] No recursion, goto, setjmp/longjmp
00087 * - **Rule 2**: [OK] Loops bounded by k_harq_soft_buffer_size
00088 * - **Rule 3**: [OK] Zero dynamic allocation
00089 * - **Rule 4**: [OK] Functions <= 60 lines
00090 * - **Rule 5**: [OK] Extensive validation checks
00091 *
00092 * @author Locked, Inc.
00093 * @date 2026-01-27
00094 * @copyright Copyright (c) 2026 Locked Inc.
00095 * SPDX-License-Identifier: MIT
00096 *
00097 * @since Version 1.0.0
00098 * @version 1.0.0
00099 */
00100
00101 #include "rx_harq.h"
00102
00103 #include "rx_bit_constants.h"
00104 #include "rx_check.h"
00105
00106 /** @brief HARQ FEC size constants */
00107 typedef enum : uint16_t {
00108     k_harq_fec_rate_multiplier = 2, /**< Rate 1/2: encoded length multiplier */
00109     k_harq_tail_bytes = 2, /**< Tail overhead in bytes */
00110 } harq_fec_constants_t;
00111
00112 /**
00113  * @enum harq_cfg_sentinel_t
00114  * @brief HARQ config validation sentinels -- zero values for "not configured, use default"
00115  * @details
00116  * Used to detect unconfigured (zero) values for max_combines and max_retries
00117  * fields in HARQ configuration, to fall back to project defaults.
00118  *
00119  * @invariant k_harq_zero_combines and k_harq_zero_retries equal zero, signifying
00120  * "use default config" -- these sentinels must never be changed to non-zero values.
00121  *
00122  * @code
00123  * // Checking sentinels when validating config fields:
00124  * const rx_harq_config_t config = { .max_combines = 0, .max_retries = 0 };
00125  * // max_combines == k_harq_zero_combines -> use k_harq_default_combines
00126  * // max_retries == k_harq_zero_retries -> use k_harq_default_retries
00127  * uint8_t combines = (config.max_combines > k_harq_zero_combines)
00128  *     ? config.max_combines : k_harq_default_combines;
00129  * uint8_t retries = (config.max_retries > k_harq_zero_retries)
00130  *     ? config.max_retries : k_harq_default_retries;
00131  * @endcode
00132  *
00133  * @see rx_harq_config_t HARQ configuration structure containing max_combines/max_retries
00134  * @see rx_harq_init() Applies these sentinels during initialization
00135  *
00136  * @since Version 1.0.0
00137  */
00138 typedef enum : uint8_t {
00139     k_harq_zero_combines = 0, /**< Sentinel: no max_combines configured -> fall back to default */
00140     k_harq_zero_retries = 0, /**< Sentinel: no max_retries configured -> fall back to default */
00141 } harq_cfg_sentinel_t;
00142
00143 typedef enum : uint8_t {
00144     k_harq_false = 0U,
00145     k_harq_true = 1U,
00146 } harq_bool_t;
00147
00148 /*
=====
00149  * Chase Combiner Implementation
00150  *
=====
00151  */
00152
00153 rx_err_t rx_chase_combiner_init(rx_chase_combiner_t* combiner, const uint8_t max_combines)
00154 {
00155     if (combiner == nullptr) {
00156         return k_rx_err_invalid_arg;
00157     }
00158
00159     /* Clear accumulator */
00160     for (uint32_t i = 0U; i < k_harq_soft_buffer_size; i++) {
00161         combiner->accumulated[i] = 0;
00162     }
00163
00164     combiner->expected_len = 0;
00165     combiner->count = 0;
00166     if (max_combines > k_harq_zero_combines) {
00167         combiner->max_combines = max_combines;

```

```

00168 } else {
00169     combiner->max_combines = k_harq_default_combines;
00170 }
00171 combiner->initialized = k_harq_true;
00172
00173 return k_rx_ok;
00174 }
00175
00176 rx_err_t rx_chase_combiner_deinit(rx_chase_combiner_t* combiner)
00177 {
00178     if (combiner == nullptr) {
00179         return k_rx_err_invalid_arg;
00180     }
00181
00182     combiner->initialized = k_harq_false;
00183     return k_rx_ok;
00184 }
00185
00186 rx_err_t
00187 rx_chase_combiner_add(rx_chase_combiner_t* combiner, const rx_soft_bit_t* soft_bits, uint32_t len)
00188 {
00189     if (combiner == nullptr || soft_bits == nullptr) {
00190         return k_rx_err_invalid_arg;
00191     }
00192
00193     if (combiner->initialized == k_harq_false) {
00194         return k_rx_err_invalid_state;
00195     }
00196
00197     if (len == 0) {
00198         return k_rx_err_invalid_arg;
00199     }
00200
00201     if (len > k_harq_soft_buffer_size) {
00202         return k_rx_err_invalid_size;
00203     }
00204
00205     if (combiner->count >= combiner->max_combines) {
00206         return k_rx_err_busy;
00207     }
00208
00209     /* First transmission sets the expected length */
00210     if (combiner->count == 0) {
00211         combiner->expected_len = len;
00212         /* Clear accumulator for new frame */
00213         for (uint32_t i = 0; i < len; i++) {
00214             combiner->accumulated[i] = 0;
00215         }
00216     } else if (len != combiner->expected_len) {
00217         return k_rx_err_invalid_size;
00218     }
00219
00220     /*
00221      * Element-wise addition (accumulate).
00222      *
00223      * Loop bound verification (NASA Rule 2):
00224      * - len is validated: len > 0 (line 62-64) AND len <= k_harq_soft_buffer_size (line 66-68)
00225      * - Maximum iterations: k_harq_soft_buffer_size (16400)
00226      * - Array bounds: accumulated[i] and soft_bits[i] are safe for i < len
00227      */
00228     for (uint32_t i = 0; i < len; i++) {
00229         combiner->accumulated[i] = (int16_t)(combiner->accumulated[i] + (int16_t)soft_bits[i]);
00230     }
00231
00232     combiner->count++;
00233     return k_rx_ok;
00234 }
00235
00236 rx_err_t rx_chase_combiner_combined(const rx_chase_combiner_t* combiner,
00237                                     rx_soft_bit_t* output,
00238                                     uint32_t* len)
00239 {
00240     if (combiner == nullptr || output == nullptr || len == nullptr) {
00241         return k_rx_err_invalid_arg;
00242     }
00243
00244     if (combiner->initialized == k_harq_false) {
00245         return k_rx_err_invalid_state;
00246     }
00247
00248     const bool count_zero = (bool)(combiner->count == 0);
00249     const bool len_zero = (bool)(combiner->expected_len == 0);
00250     if ((bool)((int)count_zero | (int)len_zero)) {
00251         return k_rx_err_invalid_state;
00252     }
00253
00254     /*

```

```

00255  * Clamp accumulated values to SoftBit range [-127, +127].
00256  *
00257  * Loop bound verification (NASA Rule 2):
00258  * - expected_len is set only via rx_chase_combiner_add() with validated len
00259  * - expected_len > 0 (validated at line 103-105)
00260  * - expected_len <= k_harq_soft_buffer_size (validated when set in add)
00261  * - Maximum iterations: k_harq_soft_buffer_size (16400)
00262  * - Array bounds: accumulated[i] and output[i] are safe for i < expected_len
00263  */
00264  for (uint32_t i = 0; i < combiner->expected_len; i++) {
00265      const int16_t acc = combiner->accumulated[i];
00266      if (acc > k_soft_bit_max) {
00267          output[i] = k_soft_bit_max;
00268      } else if (acc < k_soft_bit_min) {
00269          output[i] = k_soft_bit_min;
00270      } else {
00271          output[i] = (rx_soft_bit_t)acc;
00272      }
00273  }
00274
00275  *len = combiner->expected_len;
00276  return k_rx_ok;
00277 }
00278
00279 rx_err_t rx_chase_combiner_reset(rx_chase_combiner_t* combiner)
00280 {
00281     if (combiner == nullptr) {
00282         return k_rx_err_invalid_arg;
00283     }
00284
00285     if (combiner->initialized == k_harq_false) {
00286         return k_rx_err_invalid_state;
00287     }
00288
00289     /* Clear the entire accumulator to prevent stale data */
00290     for (uint32_t i = 0; i < k_harq_soft_buffer_size; i++) {
00291         combiner->accumulated[i] = 0;
00292     }
00293     combiner->count = 0;
00294     combiner->expected_len = 0;
00295
00296     return k_rx_ok;
00297 }
00298
00299 /**
00300  * @brief Check if combiner can accept more transmissions
00301  *
00302  */
00303 /*
00304 bool rx_chase_combiner_can_add(const rx_chase_combiner_t* combiner)
00305 {
00306     if (combiner == nullptr) {
00307         return false;
00308     }
00309     if (combiner->initialized == k_harq_false) {
00310         return false;
00311     }
00312     return (bool)(combiner->count < combiner->max_combines);
00313 }
00314
00315 /**
00316  * @brief Get number of transmissions combined so far
00317  *
00318  */
00319 /*
00320 uint8_t rx_chase_combiner_count(const rx_chase_combiner_t* combiner)
00321 {
00322     if (combiner == nullptr) {
00323         return 0;
00324     }
00325     return combiner->count;
00326 }
00327
00328 /*
00329  * HARQ Handle Implementation
00330  *
00331  */
00332
00333 rx_err_t rx_harq_init(rx_harq_handle_t* harq, const rx_harq_config_t* config)
00334 {
00335     if (harq == nullptr) {
00336         return k_rx_err_invalid_arg;
00337     }
00338
00339     /* Initialize state */

```

```

00340 harq->state      = k_harq_state_idle;
00341 harq->tx_sequence = 0;
00342 harq->rx_sequence = 0;
00343 harq->retry_count = 0;
00344
00345 /* Apply configuration */
00346 if (config != nullptr) {
00347     if (config->max_retries > k_harq_zero_retries) {
00348         harq->max_retries = config->max_retries;
00349     } else {
00350         harq->max_retries = k_harq_default_retries;
00351     }
00352     harq->fec_enabled = config->fec_enabled;
00353 } else {
00354     harq->max_retries = k_harq_default_retries;
00355     harq->fec_enabled = k_harq_true; /* FEC enabled by default */
00356 }
00357
00358 /* Initialize Chase Combiner */
00359 const bool has_combines =
00360     (bool)(config != nullptr && (int)(config->max_combines > k_harq_zero_combines));
00361 const uint8_t max_combines = (int)has_combines ? config->max_combines : k_harq_default_combines;
00362 (void)(rx_chase_combiner_init(&harq->combiner, max_combines));
00363 /* Post-condition: combiner_init only fails on nullptr; &harq->combiner is never null */
00364
00365 /* Initialize FEC encoder if enabled */
00366 if (harq->fec_enabled == k_harq_true) {
00367     (void)(rx_fec_encoder_init(&harq->encoder));
00368     /* Post-condition: encoder_init only fails on nullptr; &harq->encoder is never null */
00369 }
00370
00371 /* Initialize FEC decoder with survivors buffer */
00372 (void)(rx_fec_decoder_init(&harq->decoder,
00373     harq->decoder_survivors,
00374     k_harq_survivors_buffer_size));
00375 /* Post-condition: decoder_init only fails on nullptr or zero size; both are impossible here */
00376 }
00377 harq->initialized = k_harq_true;
00378 return k_rx_ok;
00379 }
00380
00381 /**
00382  * @brief Deinitialize HARQ handle and release resources
00383  *
00384  */
00385 rx_err_t rx_harq_deinit(rx_harq_handle_t* harq)
00386 {
00387     if (harq == nullptr) {
00388         return k_rx_err_invalid_arg;
00389     }
00390
00391     if (harq->fec_enabled == k_harq_true) {
00392         (void)(rx_fec_encoder_deinit(&harq->encoder));
00393         /* Post-condition: deinit only fails on nullptr; &harq->encoder is never null */
00394         (void)(rx_fec_decoder_deinit(&harq->decoder));
00395         /* Post-condition: deinit only fails on nullptr; &harq->decoder is never null */
00396     }
00397
00398     (void)(rx_chase_combiner_deinit(&harq->combiner));
00399     /* Post-condition: deinit only fails on nullptr; &harq->combiner is never null */
00400
00401     harq->initialized = k_harq_false;
00402     return k_rx_ok;
00403 }
00404
00405 /**
00406  * @brief Get current HARQ state
00407  *
00408  */
00409 rx_harq_state_t rx_harq_get_state(const rx_harq_handle_t* harq)
00410 {
00411     if (harq == nullptr) {
00412         return k_harq_state_error;
00413     }
00414     return harq->state;
00415 }
00416
00417 /**
00418  * @brief Reset HARQ state for new transmission
00419  *
00420  */
00421 rx_err_t rx_harq_reset(rx_harq_handle_t* harq)
00422 {
00423     if (harq == nullptr) {

```

```

00427     return k_rx_err_invalid_arg;
00428 }
00429
00430 if (harq->initialized == k_harq_false) {
00431     return k_rx_err_invalid_state;
00432 }
00433
00434 harq->state = k_harq_state_idle;
00435 harq->retry_count = 0;
00436
00437 (void)(rx_chase_combiner_reset(&harq->combiner));
00438 /* Post-condition: combiner_reset only fails on nullptr or uninitialized;
00439  * &harq->combiner is always non-null and initialized after rx_harq_init() */
00440
00441 return k_rx_ok;
00442 }
00443
00444 rx_err_t rx_harq_encode(const rx_harq_handle_t* harq,
00445                        const uint8_t* payload,
00446                        const uint32_t payload_len,
00447                        uint8_t* output,
00448                        const uint32_t output_size,
00449                        uint32_t* output_len)
00450 {
00451     if (harq == nullptr || payload == nullptr || output == nullptr || output_len == nullptr) {
00452         return k_rx_err_invalid_arg;
00453     }
00454
00455     if (harq->initialized == k_harq_false) {
00456         return k_rx_err_invalid_state;
00457     }
00458
00459     if (payload_len == 0 || payload_len > k_harq_max_payload) {
00460         return k_rx_err_invalid_size;
00461     }
00462
00463     /* If FEC is disabled, just copy payload */
00464     if (harq->fec_enabled == k_harq_false) {
00465         /* Validate output buffer can hold payload */
00466         if (output_size < payload_len) {
00467             return k_rx_err_invalid_size;
00468         }
00469         for (uint32_t i = 0; i < payload_len; i++) {
00470             output[i] = payload[i];
00471         }
00472         *output_len = payload_len;
00473         return k_rx_ok;
00474     }
00475
00476     /*
00477     * FEC encode: Rate 1/2 convolutional code doubles output size.
00478     * Required buffer: (payload_len * 8 + 6) * 2 bits = (payload_len * 8 + 6) / 4 bytes
00479     * Simplified worst case: payload_len * 2 + 2 bytes
00480     */
00481     const uint32_t min_output_size = (payload_len * k_harq_fec_rate_multiplier) + k_harq_tail_bytes;
00482     if (output_size < min_output_size) {
00483         return k_rx_err_invalid_size;
00484     }
00485
00486     /* FEC encode */
00487     return rx_fec_encode(&harq->encoder, payload, payload_len, output, output_len);
00488 }
00489
00490 /*
=====
00491 * Decode Helper Functions
00492 *
=====
00493 */
00494 /**
00495  * @brief Bit position constants for soft-to-hard conversion
00496  */
00497
00498 typedef enum : uint8_t {
00499     k_msb_position = k_rx_bits_per_byte - 1U, /**< Most significant bit position */
00500     k_rounding_adjustment = k_rx_bits_per_byte - 1U, /**< Ceiling division: (n + 7) / 8 */
00501     k_soft_bit_zero_thresh = 0, /**< Threshold for soft-to-hard: >= 0 is bit 1 */
00502 } bit_constants_t;
00503
00504 /**
00505  * @brief Convert combined soft bits to hard bits (non-FEC path)
00506  */
00507
00508
00509 static rx_err_t internal_soft_to_hard(const rx_soft_bit_t* soft_bits,
00510                                       const uint32_t soft_bit_count,
00511                                       const uint32_t max_output_bytes,

```

```

00512             uint8_t*      output,
00513             uint32_t*      output_len)
00514 {
00515     /* Pre-conditions: pointers are always valid from rx_harq_decode (harq->decode_buffer,
00516      * caller-validated output, caller-validated output_len) */
00517     if (max_output_bytes == 0) {
00518         return k_rx_err_invalid_arg;
00519     }
00520
00521     /* Calculate output bytes with ceiling division */
00522     uint32_t out_bytes = (soft_bit_count + k_rounding_adjustment) / k_rx_bits_per_byte;
00523     if (out_bytes > max_output_bytes) {
00524         out_bytes = max_output_bytes;
00525     }
00526     for (uint32_t i = 0U; i < out_bytes; i++) {
00527         output[i] = 0U;
00528     }
00529
00530     /*
00531      * Loop bound: i < soft_len is validated by prior check that soft_bits != nullptr.
00532      * Loop terminates when i >= soft_len OR when byte index exceeds out_bytes.
00533      * Maximum iterations: min(soft_len, out_bytes * k_rx_bits_per_byte).
00534      */
00535     for (uint32_t i = 0; i < soft_bit_count && (i / k_rx_bits_per_byte) < out_bytes; i++) {
00536         if (soft_bits[i] >= k_soft_bit_zero_thresh) {
00537             const uint32_t byte_idx = i / k_rx_bits_per_byte;
00538             const uint32_t bit_pos = k_msb_position - (i % k_rx_bits_per_byte);
00539             output[byte_idx] |= (uint8_t)(1U << bit_pos);
00540         }
00541     }
00542
00543     *output_len = out_bytes;
00544     return k_rx_ok;
00545 }
00546
00547 /**
00548  * @brief Handle FEC decode result and update HARQ state
00549  *
00550  */
00551 /*
00552 static rx_err_t internal_handle_fec_result(rx_harq_handle_t* harq, const rx_err_t result)
00553 {
00554     /* Pre-condition: only called from rx_harq_decode where harq is already validated */
00555
00556     harq->retry_count++;
00557
00558     if (result == k_rx_ok) {
00559         /* Ignore return - combiner is already validated via harq->initialized */
00560         (void)rx_chase_combiner_reset(&harq->combiner);
00561         harq->state = k_harq_state_idle;
00562         return k_rx_ok;
00563     }
00564
00565     /* Decode failed - check if we can retry */
00566     const bool can_add_more = rx_chase_combiner_can_add(&harq->combiner);
00567     const bool retries_remain = (bool)(harq->retry_count < harq->max_retries);
00568     if ((bool)((int)can_add_more & (int)retries_remain)) {
00569         harq->state = k_harq_state_combining;
00570         return k_rx_err_protocol_error; /* Need more retransmissions */
00571     }
00572
00573     /* Max retries reached */
00574     (void)rx_chase_combiner_reset(&harq->combiner);
00575     harq->state = k_harq_state_error;
00576     return result;
00577 }
00578
00579 /*
=====
00580 * Decode API
00581 *
=====
00582 */
00583
00584 rx_err_t rx_harq_decode(rx_harq_handle_t* harq,
00585                        const rx_harq_decode_params_t* params,
00586                        uint8_t* output,
00587                        uint32_t* output_len)
00588 {
00589     const bool harq_null = (bool)(harq == nullptr);
00590     const bool params_null = (bool)(params == nullptr);
00591     const bool output_null = (bool)(output == nullptr);
00592     const bool output_len_null = (bool)(output_len == nullptr);
00593     if ((bool)((int)harq_null | (int)params_null | (int)output_null | (int)output_len_null)) {
00594         return k_rx_err_invalid_arg;
00595     }
00596     if (harq->initialized == k_harq_false) {

```

```

00597     return k_rx_err_invalid_state;
00598 }
00599 if (params->soft_bits == nullptr || params->soft_len == 0) {
00600     return k_rx_err_invalid_arg;
00601 }
00602
00603 /* Add soft bits to combiner */
00604 rx_err_t err = rx_chase_combiner_add(&harq->combiner, params->soft_bits, params->soft_len);
00605 const bool not_ok = (bool)(err != k_rx_ok);
00606 const bool not_busy = (bool)(err != k_rx_err_busy);
00607 if ((bool)((int)not_ok & (int)not_busy)) {
00608     return err;
00609 }
00610
00611 /* Get combined soft bits into handle's buffer (thread-safe) */
00612 uint32_t combined_len = 0U;
00613 (void)rx_chase_combiner_combined(&harq->combiner, harq->decode_buffer, &combined_len);
00614 /* Post-condition: combiner_combined only fails if count==0 or expected_len==0,
00615  * but combiner_add just succeeded (count>=1, expected_len>0) */
00616
00617 /* Non-FEC path: direct soft-to-hard conversion */
00618 if (harq->fec_enabled == k_harq_false) {
00619     err = internal_soft_to_hard(harq->decode_buffer,
00620                               combined_len,
00621                               params->expected_output_len,
00622                               output,
00623                               output_len);
00624     if (err != k_rx_ok) {
00625         return err;
00626     }
00627     harq->retry_count++;
00628     /* Ignore return - combiner is already validated via harq->initialized */
00629     (void)rx_chase_combiner_reset(&harq->combiner);
00630     harq->state = k_harq_state_idle;
00631     return k_rx_ok;
00632 }
00633
00634 /* FEC path: decode combined soft bits */
00635 const rx_fec_decode_soft_params_t decode_params = {
00636     .soft_bits = harq->decode_buffer,
00637     .soft_len = combined_len,
00638     .expected_output_len = params->expected_output_len,
00639     .output = output,
00640     .output_len = output_len,
00641 };
00642
00643 err = rx_fec_decode_soft(&harq->decoder, &decode_params);
00644
00645 return internal_handle_fec_result(harq, err);
00646 }
00647
00648 /**
00649  * @brief Get number of retries attempted
00650  *
00651  */
00652 uint8_t rx_harq_get_retry_count(const rx_harq_handle_t* harq)
00653 {
00654     if (harq == nullptr) {
00655         return 0;
00656     }
00657     return harq->retry_count;
00658 }
00659
00660 /**
00661  * @brief Check if more retries are allowed
00662  *
00663  */
00664 bool rx_harq_can_retry(const rx_harq_handle_t* harq)
00665 {
00666     if (harq == nullptr) {
00667         return false;
00668     }
00669     if (harq->initialized == k_harq_false) {
00670         return false;
00671     }
00672     const bool retries_left = (bool)(harq->retry_count < harq->max_retries);
00673     const bool can_add = rx_chase_combiner_can_add(&harq->combiner);
00674     return (bool)((int)retries_left & (int)can_add);
00675 }
00676
00677 }

```

